

```
In [94]: import numpy as np
import pandas as pd
import time

import statsmodels.api as sm
import scipy.stats as stats
from numpy.linalg import LinAlgError

from sklearn.linear_model import LinearRegression, Lasso, LassoCV, Ridge, RidgeCV
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import BaggingRegressor, RandomForestRegressor

from sklearn.preprocessing import StandardScaler, OneHotEncoder, PolynomialFeatures
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.feature_selection import RFE

from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.metrics import (mean_absolute_error, mean_squared_error, r2_score,
                             confusion_matrix, roc_curve, roc_auc_score)

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.tree import plot_tree
```

The overall objective is to explain the sale prices by a set of variables using the dataset from the previous assignment.

The variable `SalePrice` is taken as the dependent variable, while the remaining variables are explanatory.

## Task 1 Ridge and Lasso Regressions

Ridge and lasso regressions are regularization techniques used to stabilize the estimation.

### Task 1.a

- Explain in your own words the concept of lasso regression.
- Sketch a scenario where a simple linear regression fails but the lasso regression can still be estimated.
- Why do we need to maximize the objective function numerically (in contrast to ridge regression)?

Lasso regression is a technique to enhance linear regression by adding *L1 regularization*  $\lambda \sum_{j=1}^p |\beta_j|$  (key point is in using the absolute value for the

penalty to make it independent regarding to the sign). This penalty forces some regression coefficients to become exactly zero, effectively performing automatic feature selection. By doing so, Lasso helps in reducing model complexity, preventing overfitting, and identifying the most relevant predictors.

### **When simple linear regression fails but Lasso works**

Imagine a dataset where we have *many highly correlated features* (multicollinearity). In a standard linear regression, the model struggles to assign meaningful coefficients, often resulting in unstable predictions. For example, if two features are strongly correlated, their coefficients might fluctuate significantly depending on small changes in the data.

Lasso, however, selects *only one* of the *correlated features* while setting the others to zero. This leads to a simpler and more interpretable model.

Another case is when the dataset contains many *irrelevant or redundant variables*. A simple linear model would include all features, leading to *overfitting*, whereas Lasso naturally eliminates the unnecessary ones by shrinking their coefficients to zero.

### **Why we need numerical optimization in Lasso (unlike Ridge)**

Lasso regression uses *L1 regularization*, which involves an *absolute value* term in the penalty. Unlike Ridge regression (which uses L2 regularization  $\lambda \sum_{j=1}^p \beta_j^2$  and has a smooth differentiable solution), Lasso's penalty creates a *non-differentiable point at zero*. This means standard analytical solutions (like matrix inversion) cannot be applied directly.

Instead, we need specialized numerical optimization techniques, such as *coordinate descent* or *gradient-based approaches*, to find the optimal set of coefficients. These methods iteratively update weights while ensuring that some coefficients become exactly zero.

## **Task 1.b**

- For the usual regression model, the variables are rarely normalized/standardized.
- However, in the case of the lasso regression, scaling becomes crucial. Why?
- Scale your data.
- Can/should the binary variables be scaled in the same fashion?
- What about factor variables (for example, `FireplaceQu`)?

### **Why scaling is so important?**

Lasso penalizes coefficients based on their absolute magnitude. If features are not scaled, those with larger numerical ranges ( `LotArea` , `GrLivArea` , etc.) get larger penalties, leading to biased results. Standardization ensures all features have the same scale, making the regularization equally applied across all variables.

### Can/should the binary variables be scaled in the same fashion?

No, binary variables (0/1) should generally not be standardized in the same way as continuous features, because they already have a fixed range, and standardization would transform them into values centered around zero, possibly making interpretation harder. Lasso regression handles binary variables well in their raw form, without requiring transformation.

However, if a dataset contains *imbalanced binary features* (for example 95% zeros, 5% ones), some practitioners normalize them to avoid dominance of the majority class. But this is *not* a strict requirement for Lasso.

**What about factor variables (for example, `FireplaceQu` )?** Factor (categorical) variables need to be *encoded* before applying Lasso, allowing Lasso to handle them naturally by selecting or ignoring categories. The two main approaches:

1. One-Hot Encoding (preferred for Lasso actually) converts each category into a separate binary column ( `FireplaceQu_Excellent` , `FireplaceQu_Good` , etc.) and works well with Lasso because it can select or drop entire categories by setting coefficients to zero.
2. Ordinal Encoding assigns numeric values to categories based on order ( `Poor = 1` , `Fair = 2` , `Good = 3` ). Only appropriate if the categories have a meaningful ranking.

For `FireplaceQu` , which represents fireplace quality ("Good," "Fair," "Poor"), one-hot encoding is still the best choice since Lasso may eliminate less relevant categories.

```
In [34]: """
Data preprocessing:
- numerical features standartization
- categorical features encoding
- missing values processing
- feature scaling
"""

df = pd.read_csv('/content/train.csv')

# Drop columns with more than 50% missing values
threshold = 0.5 * len(df)
df = df.dropna(thresh=threshold, axis=1)
```

```

X = df.drop(columns=['Id', 'SalePrice'])
y = df['SalePrice']

num_cols = X.select_dtypes(include=['int64', 'float64']).columns.tolist()
cat_cols = X.select_dtypes(include=['object']).columns.tolist()
num_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")), # fill missing values with median
    ("scaler", StandardScaler()) # standardize numerical features
])

cat_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="constant", fill_value="None")), # fill missing values with None
    ("encoder", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
])

preprocessor = ColumnTransformer([
    ("num", num_pipeline, num_cols),
    ("cat", cat_pipeline, cat_cols)
])

X_preprocessed = preprocessor.fit_transform(X)

feature_names = (
    num_cols + list(preprocessor.named_transformers_["cat"].named_steps["encoder"].get_feature_names_out())
)
X_preprocessed_df = pd.DataFrame(X_preprocessed, columns=feature_names)

X_preprocessed_df.head()

```

Out[34]:

|   | MSSubClass | LotFrontage | LotArea   | OverallQual | OverallCond | YearBuilt | YearRemodded |
|---|------------|-------------|-----------|-------------|-------------|-----------|--------------|
| 0 | 0.073375   | -0.220875   | -0.207142 | 0.651479    | -0.517200   | 1.050994  | 0.000000     |
| 1 | -0.872563  | 0.460320    | -0.091886 | -0.071836   | 2.179628    | 0.156734  | 0.000000     |
| 2 | 0.073375   | -0.084636   | 0.073480  | 0.651479    | -0.517200   | 0.984752  | 0.000000     |
| 3 | 0.309859   | -0.447940   | -0.096897 | 0.651479    | -0.517200   | -1.863632 | 0.000000     |
| 4 | 0.073375   | 0.641972    | 0.375148  | 1.374795    | -0.517200   | 0.951632  | 0.000000     |

5 rows × 282 columns

## Task 1.c

- Run a lasso regression with  $\lambda > 0$ .
- Plot the estimated parameters as functions of  $\lambda$ .
- Which value of  $\lambda$  would you recommend?
- If it is easy to implement, determine the optimal  $\lambda$  by cross-validation.

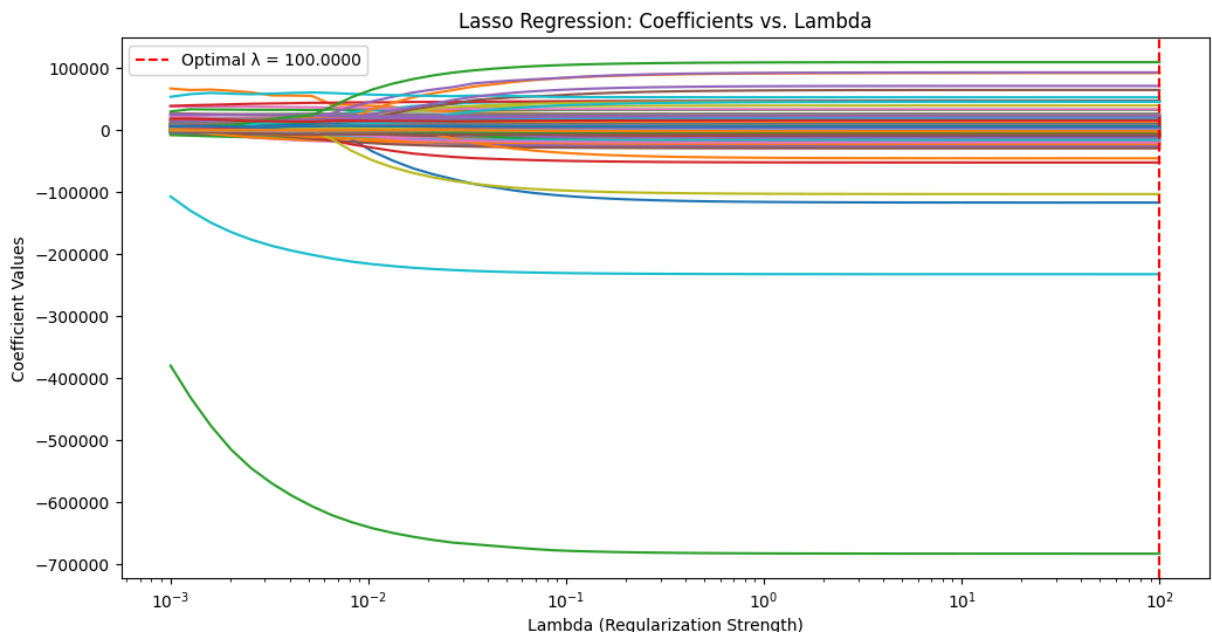
In [10]: `lambda_values = np.logspace(-3, 2, 50)`

```
# Lasso regression with cross-validation
lasso_cv = LassoCV(alphas=lambda_values, cv=5, max_iter=20000, n_jobs=-1)
lasso_cv.fit(X_preprocessed_df, y)

optimal_lambda = lasso_cv.alpha_

# Coefficient paths as a function of lambda
plt.figure(figsize=(12, 6))
plt.plot(lambda_values, lasso_cv.path(X_preprocessed_df, y, alphas=lambda_values))
plt.xscale("log")
plt.xlabel("Lambda (Regularization Strength)")
plt.ylabel("Coefficient Values")
plt.title("Lasso Regression: Coefficients vs. Lambda")
plt.axvline(optimal_lambda, color="r", linestyle="--", label=f"Optimal  $\lambda = \{optimal\_lambda\}$ ")
plt.legend()
plt.show()

print(f"Optimal lambda (defined with cross-validation) is: {optimal_lambda}")
```



Optimal lambda (defined with cross-validation) is: 100.0

In the plot, we can see how different features change as  $\lambda$  increases. When  $\lambda$  is small, most features still have large coefficients, meaning they all contribute to the model. As we increase  $\lambda$ , Lasso starts shrinking some coefficients to zero, removing the less important features. At  $\lambda = 100$ , which was chosen as the best value through cross-validation, only the most relevant features remain while the others are eliminated.

Let's check which features were retained at  $\lambda = 100$ , since they are supposed to be the ones that actually influence the sale price.

```
In [11]: # Get the coefficients for the optimal lambda
lasso_best = Lasso(alpha=optimal_lambda, max_iter=20000)
lasso_best.fit(X_preprocessed_df, y)
```

```
# Non-zero coefficients (features retained)
selected_features = X_preprocessed_df.columns[lasso_best.coef_ != 0]
selected_coefs = lasso_best.coef_[lasso_best.coef_ != 0]

retained_features_df = pd.DataFrame({'Feature': selected_features, 'Coefficient': selected_coefs})
retained_features_df = retained_features_df.sort_values(by="Coefficient", ascending=False)
retained_features_df
```

Out[11]:

|     | Feature              | Coefficient    |
|-----|----------------------|----------------|
| 65  | RoofMatl_WdShngl     | 47108.629724   |
| 50  | Neighborhood_NoRidge | 38944.244609   |
| 54  | Neighborhood_StoneBr | 38602.001351   |
| 94  | KitchenQual_Ex       | 25356.057516   |
| 13  | GrLivArea            | 24952.161443   |
| ... | ...                  | ...            |
| 47  | Neighborhood_Mitchel | -7478.755766   |
| 45  | Neighborhood_Edwards | -8826.656961   |
| 32  | MSZoning_C (all)     | -9375.527150   |
| 56  | Condition2_PosN      | -116020.356108 |
| 64  | RoofMatl_ClyTile     | -385993.298958 |

111 rows × 2 columns

## Task 1.d

- Consider the ridge regression.
- Estimate it for a fixed (and optimal, if possible) value of  $\lambda$ .
- Compare the estimated parameters and their variances for ridge regression and multiple linear regression.
- What would you expect from such a comparison?
- Remember to fit the linear regression to scaled variables.

### What would you expect from such a comparison?

Typically we expect Ridge to produce smaller, more stable coefficients, while linear regression allows coefficients to grow much larger, leading to instability. Ridge adds L2 regularization, preventing extreme coefficient values and handling multicollinearity effectively. This should result in *lower variance* in Ridge regression coefficients compared to linear regression.

In [12]: `lambda_values_ridge = np.logspace(-3, 2, 50)`

```

# Ridge regression with cross-validation
ridge_cv = RidgeCV(alphas=lambda_values_ridge, store_cv_results=True)
ridge_cv.fit(X_preprocessed_df, y)

optimal_lambda_ridge = ridge_cv.alpha_

# Regression with the optimal lambda and multiple linear regression for comparison
ridge_best = Ridge(alpha=optimal_lambda_ridge)
ridge_best.fit(X_preprocessed_df, y)
linear_reg = LinearRegression()
linear_reg.fit(X_preprocessed_df, y)

# Extract coefficients and their variances
ridge_coefs = ridge_best.coef_
linear_coefs = linear_reg.coef_
ridge_var = np.var(ridge_coefs)
linear_var = np.var(linear_coefs)

comparison_df = pd.DataFrame({
    "Feature": X_preprocessed_df.columns,
    "Linear Coefficients": linear_coefs,
    "Ridge Coefficients": ridge_coefs
}).sort_values(by="Linear Coefficients", ascending=False)

print(f"Optimal  $\lambda$  for Ridge regression: {optimal_lambda_ridge:.2f}")
print(f"Variance of coefficients (Linear regression): {linear_var:.2e}")
print(f"Variance of coefficients (Ridge regression): {ridge_var:.2e}\n")

comparison_df

```

```

Optimal  $\lambda$  for Ridge regression: 12.07
Variance of coefficients (Linear regression): 2.01e+09
Variance of coefficients (Ridge regression): 3.44e+07

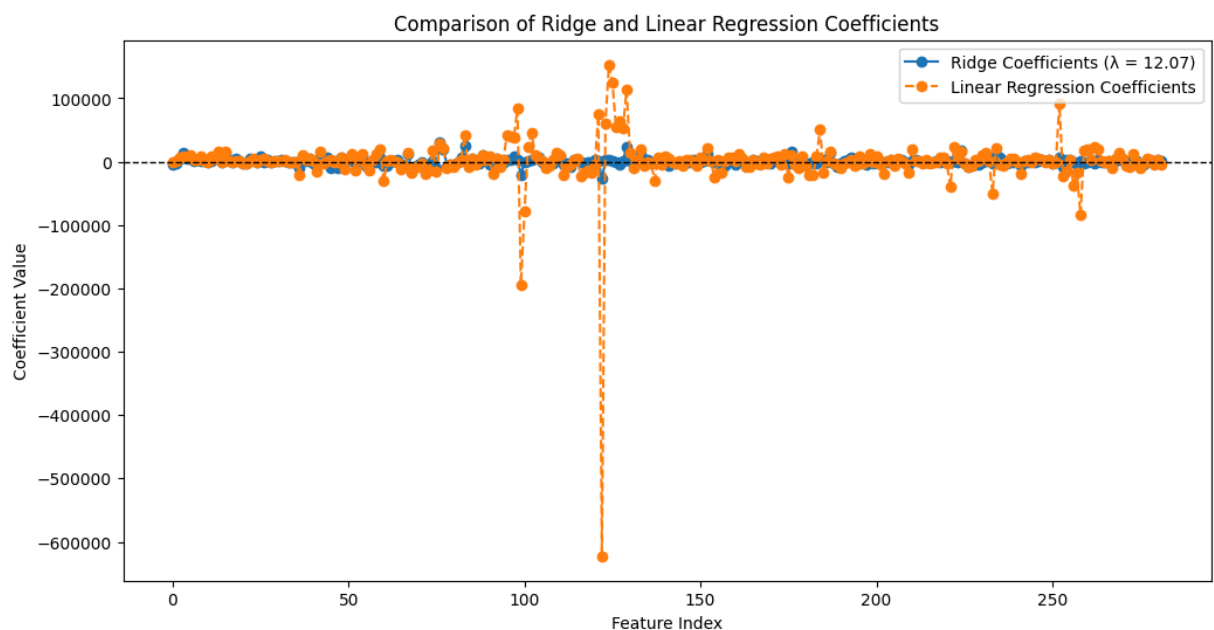
```

Out[12]:

|     | Feature          | Linear Coefficients | Ridge Coefficients |
|-----|------------------|---------------------|--------------------|
| 124 | RoofMatl_Membran | 153077.700553       | 2074.082090        |
| 125 | RoofMatl_Metal   | 124752.300998       | 695.966342         |
| 129 | RoofMatl_WdShngl | 112901.515347       | 24135.759342       |
| 252 | GarageQual_Ex    | 91721.473350        | 7114.135686        |
| 98  | Condition2_PosA  | 83545.180240        | 3708.757473        |
| ... | ...              | ...                 | ...                |
| 233 | Functional_Sev   | -50216.236009       | -3375.222651       |
| 100 | Condition2_RRAe  | -78945.073018       | -688.408012        |
| 258 | GarageCond_Ex    | -84124.844813       | 448.883272         |
| 99  | Condition2_PosN  | -194332.220828      | -20472.562057      |
| 122 | RoofMatl_ClyTile | -622566.861332      | -27053.896932      |

282 rows × 3 columns

```
In [19]: # Ridge vs Linear regression coefficients
plt.figure(figsize=(12, 6))
plt.plot(range(len(ridge_coefs)), ridge_coefs, label="Ridge Coefficients ( $\lambda$ 
plt.plot(range(len(linear_coefs)), linear_coefs, label="Linear Regression Co
plt.xlabel("Feature Index")
plt.ylabel("Coefficient Value")
plt.title("Comparison of Ridge and Linear Regression Coefficients")
plt.axhline(0, color="black", linestyle="--", linewidth=1)
plt.legend()
plt.show()
```



The variance of Ridge regression coefficients ( $3.44e+07$ ) is significantly smaller than that of linear regression ( $2.01e+09$ ), confirming that Ridge provides a *more stable and generalizable model*. The plot clearly illustrates how Ridge *shrinks coefficients*, keeping them controlled, while linear regression shows *high fluctuations* and extreme values, increasing the risk of overfitting.

These results align with expectations—*Ridge reduces variance, improves robustness, and prevents overfitting*, making it the better choice for handling datasets with high multicollinearity.

## Task 2 Regression trees

In the next step, we model `SalePrice` using regression trees.

### Task 2.a

Let `GrLivArea` be the first variable used for splitting. Write down the corresponding optimization problem and explain how the optimization works.

#### Optimizing the first split on `GrLivArea`

To build a regression tree, we need to decide *where to split* the dataset based on `GrLivArea` to best predict `SalePrice`. The goal is to create two groups where `SalePrice` is as consistent as possible within each group. The best split is the one that makes the values in each group *less scattered* (reduces variance).

To find the optimal split, we first *sort all values of* `GrLivArea` in ascending order. The possible split points are taken between consecutive values of `GrLivArea`, choosing the *midpoint* between them:

$$s = \frac{x_{(i)} + x_{(i+1)}}{2}$$

where  $x_{(i)}$  and  $x_{(i+1)}$  are these consecutive sorted values of `GrLivArea`. For each possible split, we divide the dataset into two groups: one where `GrLivArea` is less than  $s$  and another where it is greater than or equal to  $s$ .

To measure how good a split is, we calculate how much it reduces the variance of `SalePrice`:

$$\min_s \left[ \frac{n_L}{n} \text{Var}(Y_L) + \frac{n_R}{n} \text{Var}(Y_R) \right]$$

where  $n_L$  and  $n_R$  are the number of observations in each group, and  $\text{Var}(Y_L)$  and  $\text{Var}(Y_R)$  are the variances of `SalePrice` in those groups. The split that

minimizes this weighted variance is selected as the best split.

This process ensures that after the split, `SalePrice` values within each group are as similar as possible, leading to a more accurate regression tree. The same logic is applied recursively to other features, forming a complete decision tree.

## Task 2.b

Implement the optimization manually:

- Order the values of `GrLivArea`.
- Make a grid for potential split points such that every element of the grid is in the middle of two subsequent `GrLivArea` observations (i.e.,  $(x(i)+x(i+1))/2$ ).
- Compute the objective function for every element of the grid and plot it as a function of the split point.
- Compare your results with R/Python results (use this feature as a single variable and set the number of splits to 1).

```
In [23]: X_grliv = X_preprocessed_df["GrLivArea"].values
         y_price = y.values

         # Sort the values
         sorted_idx = np.argsort(X_grliv)
         X_sorted = X_grliv[sorted_idx]
         y_sorted = y_price[sorted_idx]

         # Create a grid of potential split points (midpoints between consecutive values)
         split_points = (X_sorted[:-1] + X_sorted[1:]) / 2

         # Compute the objective function (variance reduction) for each split
         def compute_variance_reduction(split, X, y):
             left_mask = X < split
             right_mask = X >= split

             if left_mask.sum() == 0 or right_mask.sum() == 0:
                 return np.inf

             var_left = np.var(y[left_mask])
             var_right = np.var(y[right_mask])

             weighted_variance = (left_mask.sum() * var_left + right_mask.sum() * var_right)
             return weighted_variance

         variance_reductions = [compute_variance_reduction(s, X_sorted, y_sorted) for s in split_points]

         # Results comparison
         plt.figure(figsize=(10, 5))
         plt.plot(split_points, variance_reductions, marker="o", linestyle="-", color="red")
         plt.xlabel("Split Point (GrLivArea)")
         plt.ylabel("Weighted Variance of SalePrice")
         plt.title("Variance Reduction vs. Split Point")
```

```
plt.grid(True)
plt.show()

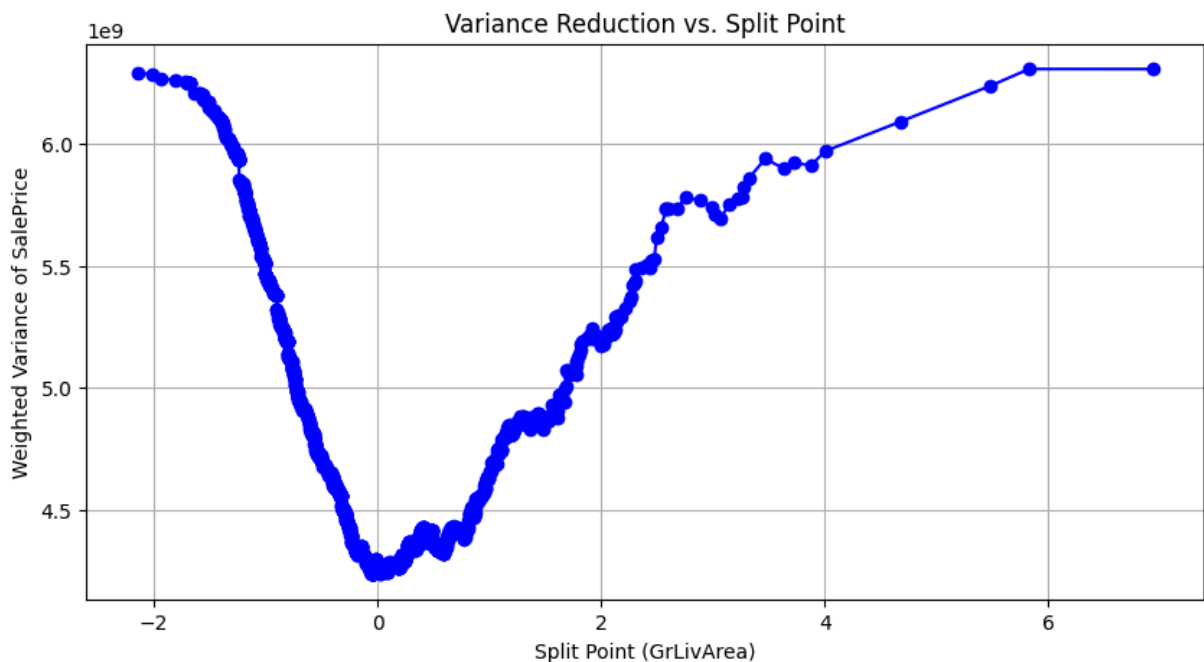
tree = DecisionTreeRegressor(max_depth=1)
tree.fit(X_grliv.reshape(-1, 1), y_price)

optimal_split_tree = tree.tree_.threshold[0]

manual_optimal_split = split_points[np.argmin(variance_reductions)]

print(f"Comparison of Manual vs sklearn's Decision Tree:")
print(f"Optimal split (manual computation): {manual_optimal_split:.2f}")
print(f"Optimal split (sklearn DecisionTreeRegressor): {optimal_split_tree:.2f}")

if np.isclose(manual_optimal_split, optimal_split_tree, atol=1e-2):
    print("The manual and sklearn approaches yield nearly identical results!")
else:
    print("The results are different. This may be due to numerical precision")
```



Comparison of Manual vs sklearn's Decision Tree:  
 Optimal split (manual computation): -0.05  
 Optimal split (sklearn DecisionTreeRegressor): -0.05

The manual and sklearn approaches yield nearly identical results!

## Task 2.c

- Consider now the variable `OverallQual`.
- It takes 10 different ordered values ranging from "very poor" to "very excellent."
- How would you determine the optimal split for this type of variable? Provide details of your ideas. Implementation is not needed.

## Finding the best split for OverallQual

Since OverallQual is an integer from 1 to 10, we can split between two consecutive values. The possible splits happen at *midpoints*: 1.5, 2.5, 3.5, ..., 9.5 and each split separates houses as  $\leq s$  or  $> s$ .

For each split, we can divide the data into two groups and calculate how spread out SalePrice is in each group (variance). The best split is the one that *reduces the variance the most* as we have done previously.

Instead of variance, we can also check *where the biggest jump in SalePrice* happens between two consecutive quality levels. If SalePrice changes suddenly at OverallQual = 6, then splitting at 5.5 or 6.5 might be the best choice.

Beyond these methods, I think, there are more advanced ways from : Beyond these methods, there are more advanced ways (basic for similar problems in terms of logic) that we can borrow:

### 1. Entropy reduction:

We can measure *how much uncertainty in SalePrice decreases* after a split. This approach, commonly used in classification trees, can be adapted for regression. Entropy quantifies uncertainty, and the best split is the one that *maximizes information gain*:

$$IG = \text{Entropy before split} - \sum \frac{n_i}{n} \text{Entropy of group}_i$$

where  $n_i$  is the number of observations in each split group. The idea is that a *good split should reduce randomness* in the data. This method works well when SalePrice is not smoothly distributed but follows distinct patterns across different OverallQual levels.

### 2. Quantile-based splitting:

We can split based on *percentiles of SalePrice* within OverallQual levels. This method is useful when the price distribution is highly skewed or has outliers (as I remember, we do have this problem when houses with OverallQual = 8 have some very expensive luxury homes, while others follow a more normal distribution). A quantile-based split can better separate such cases, ensuring that the model does not get influenced by extreme values, by grouping houses by OverallQual and calculating quartiles of SalePrice and identifying a *split point* where the distribution shifts most significantly.

### 3. Monotonic constraints:

A potential problem with standard decision trees is that they may find a split

where a higher `OverallQual` unexpectedly predicts a lower `SalePrice`, which does not make sense. Monotonic constraints prevent this by enforcing a rule that *higher quality must always correspond to higher expected prices*.

In practice, this is done by *restricting the tree-building algorithm* to ensure that splits respect the natural order of `OverallQual`. For example, in models like XGBoost or LightGBM, monotonic constraints can be applied to ensure that the tree does not create illogical splits.

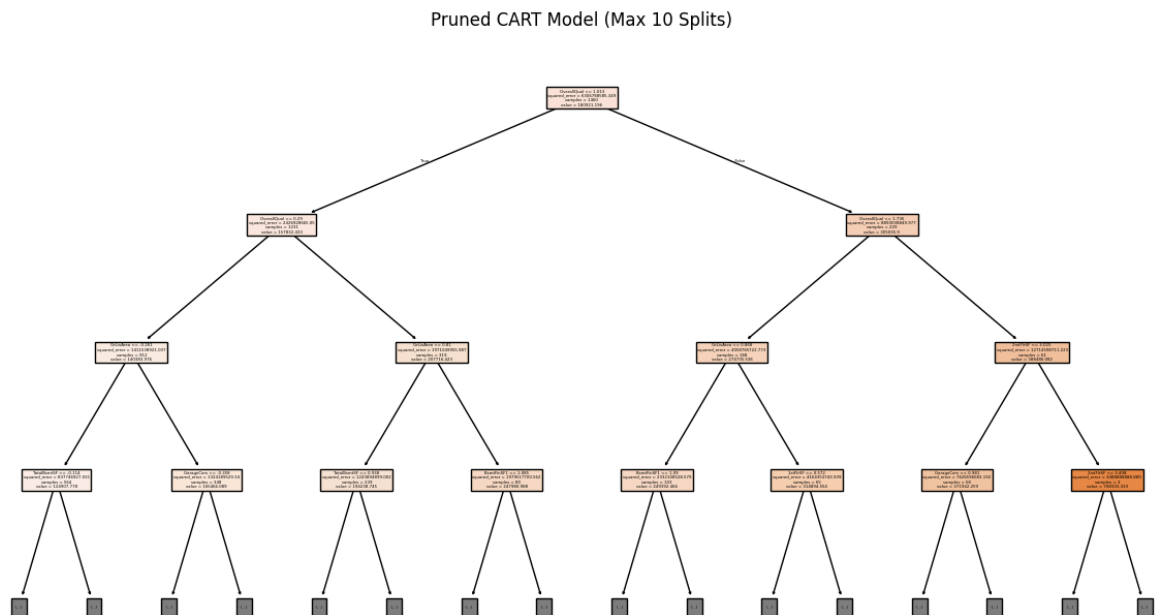
## Task 2.d

Obviously, you can get very large trees. Tree pruning helps achieve trees of a reasonable size. Fit a CART model to the data and prune it to have at most 10 splits.

```
In [3]: cart_model = DecisionTreeRegressor(random_state=42)
cart_model.fit(X_preprocessed_df, y)

pruned_cart_depth = DecisionTreeRegressor(max_depth=10, random_state=42)
pruned_cart_depth.fit(X_preprocessed_df, y)

plt.figure(figsize=(15, 8))
plot_tree(pruned_cart_depth, feature_names=X_preprocessed_df.columns, filled=True)
plt.title("Pruned CART Model (Max 10 Splits)")
plt.show()
```



Pruning a CART model to at most 10 splits prevents overfitting while keeping the tree interpretable. However, simply setting `maxdepth = 10` may not be the best pruning strategy. Instead of manually setting the parameter of max depth,

we can let the tree grow fully and then prune it dynamically using cost complexity pruning (CCP). Or we could also try improvements like boosting, bagging and random forests (from the lectures).

```
In [4]: # Fit a fully-grown tree
cart_model = DecisionTreeRegressor(random_state=42)
cart_model.fit(X_preprocessed_df, y)

# Find the cost complexity pruning path
path = cart_model.cost_complexity_pruning_path(X_preprocessed_df, y)
ccp_alphas = path.ccp_alphas
impurities = path.impurities

print(f"Possible CCP alphas: {ccp_alphas}")

# Train multiple models across all possible 'ccp_alphas' and find best one
models = []
alpha_scores = []

for alpha in ccp_alphas:
    model = DecisionTreeRegressor(random_state=42, ccp_alpha=alpha)
    scores = cross_val_score(model, X_preprocessed_df, y, cv=5, scoring='neg
    alpha_scores.append(np.mean(scores))
    models.append(model)

best_alpha_index = np.argmax(alpha_scores)
best_alpha = ccp_alphas[best_alpha_index]
print(f"Best alpha: {best_alpha}")

# Relationship between alpha and average cross-validated score
plt.figure(figsize=(8, 6))
plt.plot(ccp_alphas, alpha_scores, marker='o', drawstyle='steps-post')
plt.xlabel('alpha')
plt.ylabel('Mean CV (negative MSE)')
plt.title('Effect of alpha on CV MSE')
plt.show()

# Retrain the tree using the best alpha
pruned_cart_ccp = DecisionTreeRegressor(random_state=42, ccp_alpha=best_alpha)
pruned_cart_ccp.fit(X_preprocessed_df, y)

# New pruned tree
plt.figure(figsize=(15, 8))
plot_tree(pruned_cart_ccp, feature_names=X_preprocessed_df.columns, filled=True)
plt.title(f"Pruned CART Model (ccp_alpha = {best_alpha:.5f})")
plt.show()
```

```
Possible CCP alphas: [0.00000000e+00 8.56164384e-01 3.09075342e+00 ... 3.966
04612e+08
```

```
7.33877229e+08 2.86565497e+09]
```

```
Best alpha: 11875862.567935415
```



```

mse_pruned_depth = mean_squared_error(y, y_pred_pruned_depth)
mse_pruned_ccp = mean_squared_error(y, y_pred_pruned_ccp)

print(f"Comparison of Fully-grown vs Pruned CART models:")
print(f"Fully-grown CART model MSE: {mse_full:.2f}")
print(f"Pruned CART model (Max depth = 10) MSE: {mse_pruned_depth:.2f}")
print(f"Pruned CART model CCP MSE: {mse_pruned_ccp:.2f}\n")

# Determine best model
best_model = np.argmin([mse_full, mse_pruned_depth, mse_pruned_ccp])
best_model_name = ["Fully-grown CART", "Max depth 10 pruned CART", "CCP pruned CART"]

print(f"Best model based on MSE: {best_model_name}")

```

Comparison of Fully-grown vs Pruned CART models:  
 Fully-grown CART model MSE: 0.00  
 Pruned CART model (Max depth = 10) MSE: 106800132.18  
 Pruned CART model CCP MSE: 671264028.34

Best model based on MSE: Fully-grown CART

## Task 2.e

Find the value of the complexity parameter for the tree trained in the last question:

- The complexity parameter uniquely determines the tree.
- What does it imply?
- What happens to the tree if we significantly increase or decrease the complexity parameter?
- Check this for the given tree and data.

```

In [8]: unpruned_cart = DecisionTreeRegressor(random_state=42)
unpruned_cart.fit(X_preprocessed_df, y)

# Cost-complexity pruning path
path = unpruned_cart.cost_complexity_pruning_path(X_preprocessed_df, y)
ccp_alphas, impurities = path.ccp_alphas, path.impurities

print("Possible ccp_alphas:", ccp_alphas)
print("Corresponding total impurities:", impurities)

# Train trees for each candidate ccp_alpha
trees = []
node_counts = []
split_counts = []

for alpha in ccp_alphas:
    model = DecisionTreeRegressor(random_state=42, ccp_alpha=alpha)
    model.fit(X_preprocessed_df, y)
    trees.append(model)

    n_nodes = model.tree_.node_count

```

```

n_leaves = model.tree_.n_leaves
n_splits = n_nodes - n_leaves

node_counts.append(n_nodes)
split_counts.append(n_splits)

alpha_tree_info = list(zip(ccp_alphas, node_counts, split_counts))
print("\nalpha | #Nodes | #Splits")
for info in alpha_tree_info:
    print(f"{info[0]:.5f} | {info[1]} | {info[2]}")

# Choose the smallest alpha
best_alpha = None
for alpha, splits in zip(ccp_alphas, split_counts):
    if splits <= 10:
        best_alpha = alpha
        break

if best_alpha is None:
    best_alpha = ccp_alphas[-1]

print(f"Chosen alpha: {best_alpha:.5f}")

# Retrain a final pruned model with best alpha
pruned_cart = DecisionTreeRegressor(random_state=42, ccp_alpha=best_alpha)
pruned_cart.fit(X_preprocessed_df, y)

print(f"Final pruned tree: node_count={pruned_cart.tree_.node_count}, "
      f"n_leaves={pruned_cart.tree_.n_leaves}, "
      f"splits={pruned_cart.tree_.node_count - pruned_cart.tree_.n_leaves}")

plt.figure(figsize=(15, 8))
plot_tree(pruned_cart, feature_names=X_preprocessed_df.columns, filled=True,
plt.title(f"Pruned CART Model (ccp_alpha={best_alpha:.5f})")
plt.show()

```



[illegible]

[illegible]

|           |      |      |
|-----------|------|------|
| 342.46575 | 2465 | 1232 |
| 342.46575 | 2465 | 1232 |
| 342.46575 | 2465 | 1232 |
| 342.46575 | 2465 | 1232 |
| 342.46575 | 2465 | 1232 |
| 342.46575 | 2465 | 1232 |
| 342.46575 | 2465 | 1232 |
| 342.46575 | 2465 | 1232 |
| 342.46575 | 2465 | 1232 |
| 342.46575 | 2465 | 1232 |
| 342.46575 | 2465 | 1232 |
| 342.46575 | 2465 | 1232 |
| 342.46575 | 2465 | 1232 |
| 342.46575 | 2465 | 1232 |
| 370.41096 | 2463 | 1231 |
| 394.93151 | 2461 | 1230 |
| 395.02603 | 2459 | 1229 |
| 405.18904 | 2457 | 1228 |
| 412.10046 | 2453 | 1226 |
| 412.10046 | 2453 | 1226 |
| 412.10046 | 2449 | 1224 |
| 412.10046 | 2449 | 1224 |
| 412.10046 | 2447 | 1223 |
| 431.64954 | 2445 | 1222 |
| 456.62100 | 2443 | 1221 |
| 456.62100 | 2441 | 1220 |
| 456.62100 | 2439 | 1219 |
| 456.62100 | 2431 | 1215 |
| 456.62100 | 2431 | 1215 |
| 456.62100 | 2431 | 1215 |
| 456.62100 | 2431 | 1215 |
| 456.62100 | 2429 | 1214 |
| 468.80137 | 2427 | 1213 |
| 480.02283 | 2425 | 1212 |
| 484.42922 | 2423 | 1211 |
| 493.15068 | 2421 | 1210 |
| 524.40068 | 2419 | 1209 |
| 578.76712 | 2415 | 1207 |
| 578.76712 | 2415 | 1207 |
| 657.53425 | 2409 | 1204 |
| 657.53425 | 2409 | 1204 |
| 657.53425 | 2409 | 1204 |
| 659.81735 | 2407 | 1203 |
| 671.23288 | 2401 | 1200 |
| 671.23288 | 2401 | 1200 |
| 671.23288 | 2401 | 1200 |
| 684.93151 | 2399 | 1199 |
| 699.20091 | 2397 | 1198 |
| 701.53436 | 2395 | 1197 |
| 713.47032 | 2391 | 1195 |
| 713.47032 | 2391 | 1195 |
| 713.47032 | 2385 | 1192 |
| 713.47032 | 2385 | 1192 |
| 713.47032 | 2385 | 1192 |
| 713.47032 | 2381 | 1190 |
| 713.47032 | 2381 | 1190 |
| 713.47032 | 2379 | 1189 |

[illegible]

[illegible]

|            |      |      |
|------------|------|------|
| 2080.89897 | 2153 | 1076 |
| 2109.95434 | 2151 | 1075 |
| 2140.41096 | 2117 | 1058 |
| 2140.41096 | 2117 | 1058 |
| 2140.41096 | 2117 | 1058 |
| 2140.41096 | 2117 | 1058 |
| 2140.41096 | 2117 | 1058 |
| 2140.41096 | 2117 | 1058 |
| 2140.41096 | 2117 | 1058 |
| 2140.41096 | 2117 | 1058 |
| 2140.41096 | 2117 | 1058 |
| 2140.41096 | 2117 | 1058 |
| 2140.41096 | 2117 | 1058 |
| 2140.41096 | 2117 | 1058 |
| 2140.41096 | 2117 | 1058 |
| 2140.41096 | 2117 | 1058 |
| 2140.41096 | 2117 | 1058 |
| 2140.41096 | 2117 | 1058 |
| 2140.41096 | 2117 | 1058 |
| 2140.41096 | 2117 | 1058 |
| 2140.41096 | 2117 | 1058 |
| 2140.41096 | 2117 | 1058 |
| 2140.41096 | 2117 | 1058 |
| 2210.04566 | 2115 | 1057 |
| 2210.04566 | 2113 | 1056 |
| 2274.76992 | 2111 | 1055 |
| 2283.10502 | 2109 | 1054 |
| 2311.64384 | 2107 | 1053 |
| 2311.64384 | 2105 | 1052 |
| 2311.64384 | 2103 | 1051 |
| 2311.64384 | 2101 | 1050 |
| 2337.89954 | 2099 | 1049 |
| 2411.52968 | 2097 | 1048 |
| 2411.52968 | 2095 | 1047 |
| 2427.13704 | 2093 | 1046 |
| 2468.32192 | 2091 | 1045 |
| 2474.31507 | 2089 | 1044 |
| 2496.57534 | 2083 | 1041 |
| 2496.57534 | 2083 | 1041 |
| 2496.57534 | 2083 | 1041 |
| 2558.30548 | 2081 | 1040 |
| 2589.89726 | 2079 | 1039 |
| 2678.22489 | 2077 | 1038 |
| 2740.86758 | 2075 | 1037 |
| 2796.80365 | 2073 | 1036 |
| 2853.88128 | 2071 | 1035 |
| 2853.88128 | 2069 | 1034 |
| 2853.88128 | 2067 | 1033 |
| 2853.88128 | 2065 | 1032 |
| 2853.88128 | 2063 | 1031 |
| 2853.88128 | 2059 | 1029 |
| 2853.88128 | 2059 | 1029 |
| 2877.28311 | 2057 | 1028 |
| 2878.42466 | 2055 | 1027 |
| 2969.17808 | 2053 | 1026 |
| 2986.37295 | 2051 | 1025 |
| 3065.45988 | 2049 | 1024 |
| 3082.19178 | 2019 | 1009 |
| 3082.19178 | 2019 | 1009 |
| 3082.19178 | 2019 | 1009 |

|            |      |      |
|------------|------|------|
| 3082.19178 | 2019 | 1009 |
| 3082.19178 | 2019 | 1009 |
| 3082.19178 | 2019 | 1009 |
| 3082.19178 | 2019 | 1009 |
| 3082.19178 | 2019 | 1009 |
| 3082.19178 | 2019 | 1009 |
| 3082.19178 | 2019 | 1009 |
| 3082.19178 | 2019 | 1009 |
| 3082.19178 | 2019 | 1009 |
| 3082.19178 | 2019 | 1009 |
| 3082.19178 | 2019 | 1009 |
| 3082.19178 | 2019 | 1009 |
| 3210.61644 | 2017 | 1008 |
| 3210.61644 | 2015 | 1007 |
| 3242.44144 | 2013 | 1006 |
| 3257.01575 | 2011 | 1005 |
| 3287.67123 | 2009 | 1004 |
| 3291.09589 | 2007 | 1003 |
| 3453.19635 | 2005 | 1002 |
| 3453.19635 | 2003 | 1001 |
| 3453.19635 | 2001 | 1000 |
| 3506.84932 | 1999 | 999  |
| 3567.35160 | 1997 | 998  |
| 3617.29452 | 1995 | 997  |
| 3708.90411 | 1993 | 996  |
| 3729.45205 | 1989 | 994  |
| 3729.45205 | 1989 | 994  |
| 3774.25799 | 1987 | 993  |
| 3808.46758 | 1985 | 992  |
| 3837.89954 | 1983 | 991  |
| 3867.57991 | 1981 | 990  |
| 3906.67808 | 1979 | 989  |
| 3945.00057 | 1977 | 988  |
| 3958.90411 | 1971 | 985  |
| 3958.90411 | 1971 | 985  |
| 3958.90411 | 1971 | 985  |
| 3973.74429 | 1969 | 984  |
| 3978.08219 | 1967 | 983  |
| 3979.59475 | 1965 | 982  |
| 4076.19863 | 1963 | 981  |
| 4195.20548 | 1947 | 973  |
| 4195.20548 | 1947 | 973  |
| 4195.20548 | 1947 | 973  |
| 4195.20548 | 1947 | 973  |
| 4195.20548 | 1947 | 973  |
| 4195.20548 | 1947 | 973  |
| 4195.20548 | 1947 | 973  |
| 4195.20548 | 1947 | 973  |
| 4195.20548 | 1947 | 973  |
| 4280.82192 | 1943 | 971  |
| 4280.82192 | 1943 | 971  |
| 4388.12785 | 1941 | 970  |
| 4415.36204 | 1939 | 969  |
| 4474.88584 | 1937 | 968  |
| 4505.18116 | 1935 | 967  |
| 4540.02774 | 1933 | 966  |
| 4630.24966 | 1931 | 965  |

|            |      |     |
|------------|------|-----|
| 4675.79909 | 1929 | 964 |
| 4688.35616 | 1927 | 963 |
| 4815.92466 | 1925 | 962 |
| 4823.05936 | 1921 | 960 |
| 4823.05936 | 1921 | 960 |
| 4823.05936 | 1917 | 958 |
| 4823.05936 | 1917 | 958 |
| 4823.05936 | 1915 | 957 |
| 4823.05936 | 1913 | 956 |
| 4994.29224 | 1911 | 955 |
| 5124.42922 | 1909 | 954 |
| 5179.79452 | 1907 | 953 |
| 5208.90411 | 1903 | 951 |
| 5208.90411 | 1903 | 951 |
| 5219.50685 | 1901 | 950 |
| 5434.93151 | 1899 | 949 |
| 5479.45205 | 1875 | 937 |
| 5479.45205 | 1875 | 937 |
| 5479.45205 | 1875 | 937 |
| 5479.45205 | 1875 | 937 |
| 5479.45205 | 1875 | 937 |
| 5479.45205 | 1875 | 937 |
| 5479.45205 | 1875 | 937 |
| 5479.45205 | 1875 | 937 |
| 5479.45205 | 1875 | 937 |
| 5479.45205 | 1875 | 937 |
| 5479.45205 | 1875 | 937 |
| 5479.45205 | 1875 | 937 |
| 5479.45205 | 1875 | 937 |
| 5479.45205 | 1875 | 937 |
| 5479.45205 | 1875 | 937 |
| 5593.60731 | 1873 | 936 |
| 5593.60731 | 1869 | 934 |
| 5593.60731 | 1869 | 934 |
| 5593.60731 | 1867 | 933 |
| 5593.60731 | 1865 | 932 |
| 5593.60731 | 1863 | 931 |
| 5699.54338 | 1861 | 930 |
| 5707.76256 | 1859 | 929 |
| 5756.84932 | 1857 | 928 |
| 5813.26849 | 1855 | 927 |
| 5844.74886 | 1853 | 926 |
| 5917.80822 | 1851 | 925 |
| 6332.19178 | 1849 | 924 |
| 6373.49178 | 1847 | 923 |
| 6421.23288 | 1845 | 922 |
| 6421.23288 | 1843 | 921 |
| 6635.83562 | 1841 | 920 |
| 6712.32877 | 1839 | 919 |
| 6856.44977 | 1837 | 918 |
| 6934.93151 | 1829 | 914 |
| 6934.93151 | 1829 | 914 |
| 6934.93151 | 1829 | 914 |
| 6934.93151 | 1829 | 914 |
| 6992.00913 | 1827 | 913 |
| 7118.96256 | 1825 | 912 |
| 7152.73973 | 1823 | 911 |
| 7246.57534 | 1821 | 910 |
| 7305.93607 | 1815 | 907 |

|             |      |     |
|-------------|------|-----|
| 7305.93607  | 1815 | 907 |
| 7305.93607  | 1815 | 907 |
| 7305.93607  | 1813 | 906 |
| 7305.93607  | 1811 | 905 |
| 7458.90411  | 1809 | 904 |
| 7726.88356  | 1807 | 903 |
| 7769.69178  | 1805 | 902 |
| 7813.92694  | 1803 | 901 |
| 7947.48858  | 1801 | 900 |
| 8136.69164  | 1799 | 899 |
| 8150.97032  | 1797 | 898 |
| 8227.73973  | 1795 | 897 |
| 8247.71689  | 1793 | 896 |
| 8247.71689  | 1791 | 895 |
| 8247.71689  | 1789 | 894 |
| 8356.73516  | 1787 | 893 |
| 8390.41096  | 1785 | 892 |
| 8416.43836  | 1783 | 891 |
| 8527.43151  | 1781 | 890 |
| 8561.64384  | 1755 | 877 |
| 8561.64384  | 1755 | 877 |
| 8561.64384  | 1755 | 877 |
| 8561.64384  | 1755 | 877 |
| 8561.64384  | 1755 | 877 |
| 8561.64384  | 1755 | 877 |
| 8561.64384  | 1755 | 877 |
| 8561.64384  | 1755 | 877 |
| 8561.64384  | 1755 | 877 |
| 8561.64384  | 1755 | 877 |
| 8561.64384  | 1755 | 877 |
| 8561.64384  | 1755 | 877 |
| 8561.64384  | 1755 | 877 |
| 8595.92466  | 1753 | 876 |
| 8840.18265  | 1751 | 875 |
| 8907.53425  | 1749 | 874 |
| 9012.63699  | 1747 | 873 |
| 9061.64384  | 1745 | 872 |
| 9107.02055  | 1739 | 869 |
| 9132.42009  | 1737 | 868 |
| 9246.57534  | 1733 | 866 |
| 9246.57534  | 1733 | 866 |
| 9439.21233  | 1731 | 865 |
| 9631.84932  | 1729 | 864 |
| 9767.40868  | 1727 | 863 |
| 9838.90411  | 1725 | 862 |
| 9986.30137  | 1723 | 861 |
| 10155.55068 | 1721 | 860 |
| 10244.26992 | 1719 | 859 |
| 10302.51142 | 1717 | 858 |
| 10359.58904 | 1707 | 853 |
| 10359.58904 | 1707 | 853 |
| 10359.58904 | 1707 | 853 |
| 10359.58904 | 1707 | 853 |
| 10359.58904 | 1707 | 853 |
| 10402.39726 | 1705 | 852 |
| 10571.91781 | 1703 | 851 |

|             |  |      |  |     |
|-------------|--|------|--|-----|
| 10608.21918 |  | 1701 |  | 850 |
| 10655.50822 |  | 1699 |  | 849 |
| 10729.10959 |  | 1697 |  | 848 |
| 10938.87226 |  | 1695 |  | 847 |
| 10963.47032 |  | 1693 |  | 846 |
| 11047.26027 |  | 1691 |  | 845 |
| 11083.80856 |  | 1689 |  | 844 |
| 11150.91324 |  | 1687 |  | 843 |
| 11165.78767 |  | 1685 |  | 842 |
| 11177.52363 |  | 1683 |  | 841 |
| 11187.21461 |  | 1681 |  | 840 |
| 11415.52511 |  | 1679 |  | 839 |
| 11415.52511 |  | 1677 |  | 838 |
| 11415.52511 |  | 1675 |  | 837 |
| 11485.13592 |  | 1673 |  | 836 |
| 11509.13242 |  | 1671 |  | 835 |
| 11657.61986 |  | 1669 |  | 834 |
| 11720.89041 |  | 1667 |  | 833 |
| 11765.19452 |  | 1665 |  | 832 |
| 11835.61644 |  | 1663 |  | 831 |
| 11876.71233 |  | 1661 |  | 830 |
| 11958.35902 |  | 1659 |  | 829 |
| 12124.14384 |  | 1657 |  | 828 |
| 12328.76712 |  | 1637 |  | 818 |
| 12328.76712 |  | 1637 |  | 818 |
| 12328.76712 |  | 1637 |  | 818 |
| 12328.76712 |  | 1637 |  | 818 |
| 12328.76712 |  | 1637 |  | 818 |
| 12328.76712 |  | 1637 |  | 818 |
| 12328.76712 |  | 1637 |  | 818 |
| 12328.76712 |  | 1637 |  | 818 |
| 12328.76712 |  | 1637 |  | 818 |
| 12328.76712 |  | 1637 |  | 818 |
| 12363.01370 |  | 1635 |  | 817 |
| 12371.57534 |  | 1627 |  | 813 |
| 12371.57534 |  | 1627 |  | 813 |
| 12371.57534 |  | 1627 |  | 813 |
| 12371.57534 |  | 1627 |  | 813 |
| 12394.74886 |  | 1625 |  | 812 |
| 12585.61644 |  | 1621 |  | 810 |
| 12585.61644 |  | 1621 |  | 810 |
| 12585.61644 |  | 1617 |  | 808 |
| 12585.61644 |  | 1617 |  | 808 |
| 12705.76484 |  | 1615 |  | 807 |
| 12743.15068 |  | 1613 |  | 806 |
| 13023.40183 |  | 1609 |  | 804 |
| 13192.06621 |  | 1607 |  | 803 |
| 13377.56849 |  | 1605 |  | 802 |
| 13481.50685 |  | 1603 |  | 801 |
| 13888.23059 |  | 1601 |  | 800 |
| 14027.39726 |  | 1599 |  | 799 |
| 14065.06849 |  | 1597 |  | 798 |
| 14212.11096 |  | 1595 |  | 797 |
| 14339.18379 |  | 1593 |  | 796 |
| 14430.20548 |  | 1591 |  | 795 |
| 14469.17808 |  | 1589 |  | 794 |

|             |      |     |
|-------------|------|-----|
| 14576.48402 | 1587 | 793 |
| 14611.87215 | 1585 | 792 |
| 14809.93151 | 1583 | 791 |
| 15097.03196 | 1579 | 789 |
| 15097.03196 | 1579 | 789 |
| 15433.78995 | 1577 | 788 |
| 16109.58904 | 1575 | 787 |
| 16165.52511 | 1573 | 786 |
| 16304.79452 | 1571 | 785 |
| 16425.00000 | 1569 | 784 |
| 16438.35616 | 1567 | 783 |
| 16780.82192 | 1557 | 778 |
| 16780.82192 | 1557 | 778 |
| 16780.82192 | 1557 | 778 |
| 16780.82192 | 1557 | 778 |
| 16780.82192 | 1557 | 778 |
| 16885.84475 | 1555 | 777 |
| 17153.89555 | 1553 | 776 |
| 17391.78082 | 1551 | 775 |
| 17480.02283 | 1549 | 774 |
| 17836.75799 | 1547 | 773 |
| 18129.28082 | 1545 | 772 |
| 18220.01233 | 1543 | 771 |
| 18224.11244 | 1541 | 770 |
| 18301.53294 | 1539 | 769 |
| 18417.04718 | 1537 | 768 |
| 18493.15068 | 1535 | 767 |
| 18520.54795 | 1533 | 766 |
| 18544.52055 | 1531 | 765 |
| 18878.42466 | 1529 | 764 |
| 19239.72603 | 1527 | 763 |
| 19263.69863 | 1523 | 761 |
| 19263.69863 | 1523 | 761 |
| 19292.23744 | 1521 | 760 |
| 19292.23744 | 1519 | 759 |
| 19292.23744 | 1517 | 758 |
| 20515.31756 | 1515 | 757 |
| 20804.79452 | 1513 | 756 |
| 20804.79452 | 1511 | 755 |
| 21114.15525 | 1509 | 754 |
| 21332.25219 | 1507 | 753 |
| 21476.21952 | 1505 | 752 |
| 21728.82420 | 1503 | 751 |
| 21917.80822 | 1491 | 745 |
| 21917.80822 | 1491 | 745 |
| 21917.80822 | 1491 | 745 |
| 21917.80822 | 1491 | 745 |
| 21917.80822 | 1491 | 745 |
| 21917.80822 | 1491 | 745 |
| 21978.69102 | 1489 | 744 |
| 22232.17264 | 1487 | 743 |
| 22374.42922 | 1485 | 742 |
| 22438.40183 | 1483 | 741 |
| 22438.86986 | 1481 | 740 |
| 22469.17808 | 1479 | 739 |
| 22603.31050 | 1477 | 738 |

|             |      |     |
|-------------|------|-----|
| 22848.02192 | 1475 | 737 |
| 23378.99543 | 1473 | 736 |
| 23378.99543 | 1471 | 735 |
| 23579.08818 | 1469 | 734 |
| 23592.46575 | 1467 | 733 |
| 23863.67352 | 1465 | 732 |
| 24333.33333 | 1463 | 731 |
| 24743.15068 | 1459 | 729 |
| 24743.15068 | 1459 | 729 |
| 24932.07763 | 1457 | 728 |
| 24965.75342 | 1455 | 727 |
| 25035.10274 | 1453 | 726 |
| 25171.23288 | 1451 | 725 |
| 25328.76712 | 1449 | 724 |
| 25684.93151 | 1447 | 723 |
| 25684.93151 | 1445 | 722 |
| 25710.95890 | 1443 | 721 |
| 25871.31279 | 1441 | 720 |
| 25945.07397 | 1439 | 719 |
| 26031.89132 | 1437 | 718 |
| 26040.54795 | 1435 | 717 |
| 26139.26941 | 1433 | 716 |
| 26237.67123 | 1431 | 715 |
| 26315.12557 | 1429 | 714 |
| 27174.65753 | 1427 | 713 |
| 27425.79909 | 1425 | 712 |
| 27555.10274 | 1423 | 711 |
| 27625.57078 | 1421 | 710 |
| 27625.57078 | 1419 | 709 |
| 27739.72603 | 1413 | 706 |
| 27739.72603 | 1413 | 706 |
| 27739.72603 | 1413 | 706 |
| 28312.69920 | 1411 | 705 |
| 28986.30137 | 1409 | 704 |
| 29194.61952 | 1407 | 703 |
| 29223.74429 | 1405 | 702 |
| 29223.74429 | 1403 | 701 |
| 29223.74429 | 1401 | 700 |
| 30310.35959 | 1399 | 699 |
| 30329.90868 | 1397 | 698 |
| 30382.03151 | 1395 | 697 |
| 30416.66667 | 1393 | 696 |
| 30444.00828 | 1391 | 695 |
| 30907.53425 | 1387 | 693 |
| 30907.53425 | 1387 | 693 |
| 30913.23470 | 1385 | 692 |
| 31255.70776 | 1383 | 691 |
| 31400.10437 | 1381 | 690 |
| 31521.11872 | 1379 | 689 |
| 31561.64384 | 1377 | 688 |
| 31669.21644 | 1375 | 687 |
| 32106.16438 | 1373 | 686 |
| 32219.17808 | 1371 | 685 |
| 32487.67123 | 1369 | 684 |
| 32516.06301 | 1367 | 683 |
| 32603.31050 | 1365 | 682 |

|             |      |     |
|-------------|------|-----|
| 32797.08904 | 1363 | 681 |
| 32990.86758 | 1361 | 680 |
| 33029.45205 | 1359 | 679 |
| 33139.23596 | 1357 | 678 |
| 33150.68493 | 1355 | 677 |
| 33188.71301 | 1353 | 676 |
| 33375.47397 | 1351 | 675 |
| 34042.80822 | 1349 | 674 |
| 34246.57534 | 1345 | 672 |
| 34246.57534 | 1345 | 672 |
| 34541.09589 | 1343 | 671 |
| 34960.04566 | 1341 | 670 |
| 34960.04566 | 1339 | 669 |
| 35470.08813 | 1337 | 668 |
| 36001.71233 | 1335 | 667 |
| 36172.94521 | 1333 | 666 |
| 36749.61678 | 1331 | 665 |
| 36781.10731 | 1329 | 664 |
| 37012.91239 | 1327 | 663 |
| 37114.72603 | 1325 | 662 |
| 37219.39212 | 1323 | 661 |
| 37756.84932 | 1321 | 660 |
| 38015.41096 | 1319 | 659 |
| 38584.47489 | 1317 | 658 |
| 39069.63470 | 1315 | 657 |
| 39069.63470 | 1313 | 656 |
| 39298.08098 | 1311 | 655 |
| 39650.43025 | 1309 | 654 |
| 40042.80240 | 1307 | 653 |
| 41210.04566 | 1305 | 652 |
| 41432.87671 | 1303 | 651 |
| 41494.48250 | 1299 | 649 |
| 41561.71994 | 1297 | 648 |
| 41565.00007 | 1295 | 647 |
| 41609.58904 | 1293 | 646 |
| 41863.29909 | 1291 | 645 |
| 42195.20548 | 1287 | 643 |
| 42195.20548 | 1287 | 643 |
| 43164.95434 | 1285 | 642 |
| 43202.05479 | 1283 | 641 |
| 43407.53425 | 1281 | 640 |
| 43502.00702 | 1279 | 639 |
| 44033.05936 | 1277 | 638 |
| 44200.91324 | 1275 | 637 |
| 44753.42466 | 1273 | 636 |
| 45211.18721 | 1271 | 635 |
| 45291.09589 | 1265 | 632 |
| 45291.09589 | 1265 | 632 |
| 45291.09589 | 1265 | 632 |
| 45662.10046 | 1263 | 631 |
| 45781.92694 | 1261 | 630 |
| 45875.34247 | 1259 | 629 |
| 45923.42466 | 1257 | 628 |
| 46082.19178 | 1255 | 627 |
| 46579.90868 | 1253 | 626 |
| 46579.90868 | 1251 | 625 |

|             |      |     |
|-------------|------|-----|
| 47014.26941 | 1249 | 624 |
| 47137.32877 | 1247 | 623 |
| 47376.33599 | 1245 | 622 |
| 47973.74429 | 1243 | 621 |
| 48159.24658 | 1241 | 620 |
| 48442.92237 | 1239 | 619 |
| 48555.86484 | 1237 | 618 |
| 48706.24049 | 1235 | 617 |
| 48942.69406 | 1233 | 616 |
| 49140.21918 | 1231 | 615 |
| 49472.65982 | 1229 | 614 |
| 49578.99543 | 1227 | 613 |
| 49671.80365 | 1225 | 612 |
| 50342.46575 | 1223 | 611 |
| 50363.44178 | 1221 | 610 |
| 50620.93196 | 1219 | 609 |
| 50943.56450 | 1217 | 608 |
| 50959.06719 | 1215 | 607 |
| 51405.41422 | 1213 | 606 |
| 51641.24055 | 1211 | 605 |
| 51961.44121 | 1209 | 604 |
| 52278.53881 | 1207 | 603 |
| 52602.73973 | 1205 | 602 |
| 52972.24966 | 1203 | 601 |
| 53260.27397 | 1201 | 600 |
| 53510.27397 | 1195 | 597 |
| 53510.27397 | 1195 | 597 |
| 53510.27397 | 1195 | 597 |
| 53704.33790 | 1193 | 596 |
| 53754.56621 | 1191 | 595 |
| 53878.49600 | 1189 | 594 |
| 54042.30183 | 1187 | 593 |
| 54347.47953 | 1185 | 592 |
| 54430.26474 | 1183 | 591 |
| 54638.98402 | 1181 | 590 |
| 55206.25000 | 1179 | 589 |
| 55479.45205 | 1177 | 588 |
| 56276.25571 | 1175 | 587 |
| 56769.04966 | 1173 | 586 |
| 57876.71233 | 1169 | 584 |
| 57876.71233 | 1169 | 584 |
| 58177.01199 | 1167 | 583 |
| 58604.45205 | 1165 | 582 |
| 59337.18116 | 1163 | 581 |
| 60388.12785 | 1161 | 580 |
| 62157.53425 | 1159 | 579 |
| 62428.65297 | 1157 | 578 |
| 62569.00685 | 1155 | 577 |
| 64055.36530 | 1153 | 576 |
| 64390.69635 | 1151 | 575 |
| 64640.36712 | 1149 | 574 |
| 65392.27808 | 1147 | 573 |
| 65753.42466 | 1145 | 572 |
| 66302.51142 | 1143 | 571 |
| 66453.76712 | 1141 | 570 |
| 66575.34247 | 1139 | 569 |

|             |      |     |
|-------------|------|-----|
| 66603.08219 | 1137 | 568 |
| 67226.35356 | 1135 | 567 |
| 67936.64384 | 1133 | 566 |
| 67936.64384 | 1131 | 565 |
| 68085.61644 | 1129 | 564 |
| 68521.68950 | 1127 | 563 |
| 69920.09132 | 1125 | 562 |
| 70525.49315 | 1123 | 561 |
| 70595.59635 | 1121 | 560 |
| 70910.03136 | 1119 | 559 |
| 71347.03196 | 1117 | 558 |
| 71918.94977 | 1115 | 557 |
| 72337.89954 | 1113 | 556 |
| 74507.70548 | 1111 | 555 |
| 75087.74075 | 1109 | 554 |
| 76041.66667 | 1107 | 553 |
| 77054.79452 | 1105 | 552 |
| 77086.90068 | 1103 | 551 |
| 77626.92977 | 1101 | 550 |
| 78139.26941 | 1099 | 549 |
| 78660.10274 | 1097 | 548 |
| 80479.59475 | 1095 | 547 |
| 80771.68950 | 1093 | 546 |
| 81095.44777 | 1091 | 545 |
| 81508.27626 | 1089 | 544 |
| 82199.03607 | 1087 | 543 |
| 82277.39726 | 1083 | 541 |
| 82277.39726 | 1083 | 541 |
| 84083.07241 | 1081 | 540 |
| 84146.40411 | 1079 | 539 |
| 84177.97397 | 1077 | 538 |
| 84952.91096 | 1075 | 537 |
| 86605.90573 | 1073 | 536 |
| 87671.23288 | 1071 | 535 |
| 87783.56164 | 1069 | 534 |
| 88307.24070 | 1067 | 533 |
| 88613.92694 | 1065 | 532 |
| 89055.36530 | 1063 | 531 |
| 90432.36301 | 1061 | 530 |
| 90969.54353 | 1059 | 529 |
| 91192.69406 | 1057 | 528 |
| 92060.50228 | 1055 | 527 |
| 92064.57926 | 1053 | 526 |
| 92932.42009 | 1051 | 525 |
| 94392.12329 | 1049 | 524 |
| 94563.35616 | 1047 | 523 |
| 94684.93151 | 1045 | 522 |
| 95636.74168 | 1043 | 521 |
| 95653.98350 | 1039 | 519 |
| 95936.45548 | 1037 | 518 |
| 97083.75651 | 1035 | 517 |
| 97847.34349 | 1033 | 516 |
| 98776.25571 | 1031 | 515 |
| 99170.40254 | 1029 | 514 |
| 99251.44597 | 1027 | 513 |
| 99419.17808 | 1025 | 512 |

|              |      |     |
|--------------|------|-----|
| 104000.19569 | 1023 | 511 |
| 104023.97260 | 1021 | 510 |
| 104330.59361 | 1019 | 509 |
| 104880.13699 | 1017 | 508 |
| 105488.01370 | 1015 | 507 |
| 106042.07436 | 1013 | 506 |
| 106082.19178 | 1011 | 505 |
| 106767.26598 | 1009 | 504 |
| 106860.16886 | 1007 | 503 |
| 109323.63014 | 1005 | 502 |
| 109500.00000 | 1003 | 501 |
| 109529.05923 | 999  | 499 |
| 109701.43382 | 997  | 498 |
| 111428.29500 | 995  | 497 |
| 113637.04338 | 993  | 496 |
| 113652.03299 | 991  | 495 |
| 116050.41857 | 989  | 494 |
| 116645.54795 | 987  | 493 |
| 116679.45205 | 985  | 492 |
| 117208.90411 | 983  | 491 |
| 117223.01500 | 981  | 490 |
| 120557.72994 | 979  | 489 |
| 120576.48402 | 977  | 488 |
| 120776.25571 | 975  | 487 |
| 121666.66667 | 973  | 486 |
| 122264.55479 | 971  | 485 |
| 123331.57710 | 969  | 484 |
| 124110.95890 | 967  | 483 |
| 124298.63014 | 965  | 482 |
| 124315.06849 | 963  | 481 |
| 124405.39932 | 961  | 480 |
| 124891.98016 | 959  | 479 |
| 124934.93151 | 957  | 478 |
| 125022.83105 | 955  | 477 |
| 125022.83105 | 953  | 476 |
| 127234.80308 | 951  | 475 |
| 128110.73059 | 949  | 474 |
| 128626.49829 | 947  | 473 |
| 129324.20091 | 945  | 472 |
| 129853.62035 | 943  | 471 |
| 130222.60274 | 941  | 470 |
| 130372.94521 | 939  | 469 |
| 131963.47032 | 937  | 468 |
| 133568.87671 | 935  | 467 |
| 134246.57534 | 933  | 466 |
| 134915.62521 | 931  | 465 |
| 135173.16450 | 929  | 464 |
| 135619.86301 | 927  | 463 |
| 135924.65753 | 925  | 462 |
| 137005.56507 | 923  | 461 |
| 137724.40899 | 921  | 460 |
| 138212.54892 | 919  | 459 |
| 138986.31849 | 917  | 458 |
| 139840.18265 | 915  | 457 |
| 139840.18265 | 913  | 456 |
| 142702.47958 | 911  | 455 |

|              |     |     |
|--------------|-----|-----|
| 144497.04155 | 909 | 454 |
| 147491.31238 | 907 | 453 |
| 147586.90068 | 905 | 452 |
| 148055.04359 | 903 | 451 |
| 148458.90411 | 901 | 450 |
| 148762.94949 | 899 | 449 |
| 148871.28995 | 897 | 448 |
| 149277.70000 | 895 | 447 |
| 150548.32572 | 893 | 446 |
| 150687.52594 | 891 | 445 |
| 150688.73668 | 889 | 444 |
| 151592.70630 | 887 | 443 |
| 152277.90484 | 885 | 442 |
| 156278.53881 | 883 | 441 |
| 157510.66696 | 881 | 440 |
| 158042.00913 | 879 | 439 |
| 158589.12671 | 877 | 438 |
| 160530.82192 | 875 | 437 |
| 162938.63214 | 873 | 436 |
| 163446.74658 | 871 | 435 |
| 163494.86301 | 869 | 434 |
| 163813.46488 | 867 | 433 |
| 164839.47489 | 865 | 432 |
| 164840.18265 | 863 | 431 |
| 164840.18265 | 861 | 430 |
| 166579.90868 | 859 | 429 |
| 169348.22489 | 857 | 428 |
| 169534.81735 | 855 | 427 |
| 170728.06588 | 853 | 426 |
| 170969.17808 | 851 | 425 |
| 172493.29050 | 849 | 424 |
| 172659.81735 | 847 | 423 |
| 172949.48630 | 845 | 422 |
| 173373.28767 | 843 | 421 |
| 178450.51310 | 841 | 420 |
| 180639.64992 | 839 | 419 |
| 182206.05023 | 837 | 418 |
| 184039.65122 | 835 | 417 |
| 185008.56164 | 833 | 416 |
| 188593.40945 | 831 | 415 |
| 189396.66267 | 829 | 414 |
| 189554.56447 | 827 | 413 |
| 190476.13151 | 825 | 412 |
| 190739.72603 | 823 | 411 |
| 190960.04566 | 821 | 410 |
| 191483.32603 | 819 | 409 |
| 195214.04110 | 817 | 408 |
| 197336.99376 | 815 | 407 |
| 198687.21461 | 813 | 406 |
| 201102.98315 | 811 | 405 |
| 203610.30696 | 809 | 404 |
| 203809.93151 | 807 | 403 |
| 204046.44754 | 805 | 402 |
| 204256.56393 | 803 | 401 |
| 205125.04281 | 801 | 400 |
| 206050.22831 | 799 | 399 |

|              |     |     |
|--------------|-----|-----|
| 208819.20662 | 797 | 398 |
| 209753.93836 | 795 | 397 |
| 210772.44295 | 793 | 396 |
| 211073.05936 | 791 | 395 |
| 211573.97260 | 789 | 394 |
| 214041.09589 | 787 | 393 |
| 214770.61929 | 785 | 392 |
| 218725.30295 | 783 | 391 |
| 218776.93613 | 781 | 390 |
| 221714.93803 | 779 | 389 |
| 223523.11644 | 777 | 388 |
| 223767.12329 | 775 | 387 |
| 224149.05414 | 773 | 386 |
| 226033.10502 | 771 | 385 |
| 230187.51519 | 769 | 384 |
| 231506.84932 | 767 | 383 |
| 231621.49008 | 765 | 382 |
| 232657.92564 | 763 | 381 |
| 243329.86327 | 761 | 380 |
| 244137.31653 | 759 | 379 |
| 244140.41096 | 757 | 378 |
| 244279.34247 | 755 | 377 |
| 247930.10585 | 753 | 376 |
| 252698.97260 | 751 | 375 |
| 253809.93151 | 749 | 374 |
| 256323.61294 | 745 | 372 |
| 258216.43836 | 743 | 371 |
| 259058.21918 | 741 | 370 |
| 259083.28767 | 739 | 369 |
| 259442.00913 | 737 | 368 |
| 260059.93151 | 735 | 367 |
| 264759.05034 | 733 | 366 |
| 266340.82338 | 731 | 365 |
| 267209.75651 | 729 | 364 |
| 268493.15068 | 727 | 363 |
| 270613.51233 | 725 | 362 |
| 270833.84295 | 723 | 361 |
| 271297.08904 | 721 | 360 |
| 273145.37671 | 719 | 359 |
| 273493.15068 | 717 | 358 |
| 275943.37476 | 715 | 357 |
| 276255.70776 | 713 | 356 |
| 277452.16895 | 711 | 355 |
| 278877.20157 | 709 | 354 |
| 280171.66096 | 707 | 353 |
| 281430.74581 | 705 | 352 |
| 285115.58219 | 703 | 351 |
| 286392.69406 | 701 | 350 |
| 288476.06001 | 699 | 349 |
| 291051.43408 | 697 | 348 |
| 298851.86718 | 693 | 346 |
| 299589.04110 | 691 | 345 |
| 300900.27723 | 689 | 344 |
| 301496.35562 | 687 | 343 |
| 301940.63927 | 685 | 342 |
| 304461.17445 | 683 | 341 |

|              |     |     |
|--------------|-----|-----|
| 307214.61187 | 681 | 340 |
| 307983.12842 | 679 | 339 |
| 316038.15471 | 677 | 338 |
| 319447.81664 | 675 | 337 |
| 319933.56332 | 673 | 336 |
| 320662.10046 | 671 | 335 |
| 321715.28690 | 667 | 333 |
| 323854.10959 | 665 | 332 |
| 332268.83562 | 663 | 331 |
| 332322.14897 | 661 | 330 |
| 334687.82106 | 659 | 329 |
| 335683.02892 | 657 | 328 |
| 336599.93030 | 655 | 327 |
| 336606.78575 | 651 | 325 |
| 336857.99762 | 649 | 324 |
| 339049.65753 | 647 | 323 |
| 339302.30213 | 645 | 322 |
| 342822.48858 | 643 | 321 |
| 343379.13436 | 641 | 320 |
| 351626.71233 | 639 | 319 |
| 359629.26758 | 637 | 318 |
| 362846.00747 | 635 | 317 |
| 363321.91781 | 633 | 316 |
| 365549.26665 | 631 | 315 |
| 376869.29224 | 629 | 314 |
| 377810.66536 | 627 | 313 |
| 394013.98402 | 625 | 312 |
| 397616.99875 | 623 | 311 |
| 403858.44749 | 621 | 310 |
| 407619.86301 | 619 | 309 |
| 412385.84475 | 617 | 308 |
| 414253.15068 | 615 | 307 |
| 415216.02862 | 613 | 306 |
| 417428.62035 | 611 | 305 |
| 419520.54795 | 609 | 304 |
| 421164.95434 | 607 | 303 |
| 423528.15830 | 605 | 302 |
| 425954.36767 | 603 | 301 |
| 427740.62502 | 601 | 300 |
| 427793.00836 | 599 | 299 |
| 430321.39094 | 597 | 298 |
| 433387.20184 | 595 | 297 |
| 433479.20000 | 593 | 296 |
| 433672.27250 | 589 | 294 |
| 434635.78865 | 587 | 293 |
| 440543.31620 | 585 | 292 |
| 444493.15068 | 583 | 291 |
| 446210.95890 | 581 | 290 |
| 448576.00228 | 579 | 289 |
| 453351.88356 | 577 | 288 |
| 460302.51142 | 575 | 287 |
| 470864.31656 | 573 | 286 |
| 475874.39358 | 571 | 285 |
| 483959.01826 | 569 | 284 |
| 485379.89956 | 567 | 283 |
| 489754.56621 | 565 | 282 |

|              |     |     |
|--------------|-----|-----|
| 489780.01215 | 563 | 281 |
| 493664.38356 | 561 | 280 |
| 493881.27854 | 559 | 279 |
| 494033.32719 | 557 | 278 |
| 494156.42766 | 555 | 277 |
| 512794.55860 | 553 | 276 |
| 517979.45205 | 551 | 275 |
| 532623.28767 | 549 | 274 |
| 543329.01575 | 547 | 273 |
| 546180.09016 | 545 | 272 |
| 550744.12916 | 543 | 271 |
| 575684.93151 | 541 | 270 |
| 578470.35769 | 539 | 269 |
| 586307.49773 | 537 | 268 |
| 586691.49421 | 535 | 267 |
| 592355.03017 | 533 | 266 |
| 592953.96466 | 529 | 264 |
| 600636.87966 | 527 | 263 |
| 601071.30313 | 525 | 262 |
| 615856.61393 | 523 | 261 |
| 617805.65749 | 519 | 259 |
| 619073.72911 | 517 | 258 |
| 620055.93607 | 515 | 257 |
| 622517.28623 | 513 | 256 |
| 624143.83562 | 511 | 255 |
| 626255.36204 | 509 | 254 |
| 627368.41144 | 505 | 252 |
| 627452.38095 | 503 | 251 |
| 666659.70642 | 499 | 249 |
| 669076.66365 | 497 | 248 |
| 674777.50428 | 495 | 247 |
| 675292.98464 | 493 | 246 |
| 675451.26935 | 491 | 245 |
| 679366.43836 | 489 | 244 |
| 681053.53408 | 487 | 243 |
| 682651.08699 | 485 | 242 |
| 683613.01778 | 483 | 241 |
| 690967.30032 | 481 | 240 |
| 699162.18200 | 477 | 238 |
| 701139.04979 | 475 | 237 |
| 701182.49733 | 473 | 236 |
| 701941.59056 | 471 | 235 |
| 713063.74865 | 469 | 234 |
| 716054.79452 | 467 | 233 |
| 722057.64840 | 465 | 232 |
| 723573.26904 | 463 | 231 |
| 724657.53425 | 461 | 230 |
| 727811.07306 | 459 | 229 |
| 737793.34392 | 457 | 228 |
| 744542.05993 | 455 | 227 |
| 760332.77275 | 453 | 226 |
| 767579.90868 | 451 | 225 |
| 792749.17166 | 449 | 224 |
| 793734.41781 | 447 | 223 |
| 794115.29680 | 445 | 222 |
| 810756.27854 | 443 | 221 |

|               |     |     |
|---------------|-----|-----|
| 824001.85263  | 441 | 220 |
| 827244.82294  | 435 | 217 |
| 832182.09600  | 433 | 216 |
| 850608.21918  | 431 | 215 |
| 855048.62281  | 429 | 214 |
| 861511.25245  | 427 | 213 |
| 866166.09696  | 425 | 212 |
| 869452.21787  | 423 | 211 |
| 893227.73973  | 421 | 210 |
| 920561.17427  | 419 | 209 |
| 926713.47032  | 417 | 208 |
| 926867.78574  | 415 | 207 |
| 927077.19976  | 413 | 206 |
| 928756.25465  | 411 | 205 |
| 933582.37389  | 409 | 204 |
| 947111.01327  | 407 | 203 |
| 949818.08545  | 405 | 202 |
| 952468.39272  | 403 | 201 |
| 956436.43657  | 401 | 200 |
| 960610.86106  | 399 | 199 |
| 981032.67226  | 397 | 198 |
| 995872.26886  | 393 | 196 |
| 1019187.22925 | 391 | 195 |
| 1021764.84891 | 389 | 194 |
| 1034615.76487 | 387 | 193 |
| 1056654.85540 | 385 | 192 |
| 1088883.56521 | 383 | 191 |
| 1100665.97672 | 381 | 190 |
| 1105119.97455 | 379 | 189 |
| 1146620.42653 | 377 | 188 |
| 1149498.47793 | 375 | 187 |
| 1149525.97849 | 373 | 186 |
| 1151473.23416 | 369 | 184 |
| 1163278.27789 | 367 | 183 |
| 1167809.08587 | 365 | 182 |
| 1186232.90142 | 363 | 181 |
| 1197352.81684 | 361 | 180 |
| 1214075.70939 | 359 | 179 |
| 1216289.65549 | 357 | 178 |
| 1221002.83201 | 355 | 177 |
| 1243305.86348 | 353 | 176 |
| 1253385.69472 | 351 | 175 |
| 1255495.02100 | 349 | 174 |
| 1255744.82283 | 347 | 173 |
| 1271793.24658 | 345 | 172 |
| 1275178.40694 | 343 | 171 |
| 1277812.78539 | 341 | 170 |
| 1297098.83602 | 339 | 169 |
| 1303562.16272 | 337 | 168 |
| 1308486.54716 | 335 | 167 |
| 1327150.91324 | 333 | 166 |
| 1334868.49315 | 331 | 165 |
| 1344960.25274 | 329 | 164 |
| 1356278.53881 | 327 | 163 |
| 1356287.48340 | 325 | 162 |
| 1365673.73738 | 323 | 161 |

|               |     |     |
|---------------|-----|-----|
| 1371984.04223 | 319 | 159 |
| 1399392.86959 | 317 | 158 |
| 1401341.54419 | 315 | 157 |
| 1404745.57842 | 313 | 156 |
| 1437610.52304 | 311 | 155 |
| 1438507.92318 | 307 | 153 |
| 1469371.95042 | 305 | 152 |
| 1472806.58839 | 303 | 151 |
| 1478555.17840 | 301 | 150 |
| 1525418.68537 | 299 | 149 |
| 1547732.47456 | 297 | 148 |
| 1598861.15434 | 295 | 147 |
| 1603512.92087 | 293 | 146 |
| 1605290.38451 | 291 | 145 |
| 1606736.68891 | 289 | 144 |
| 1627980.39060 | 287 | 143 |
| 1663812.13307 | 285 | 142 |
| 1737847.35812 | 283 | 141 |
| 1747898.64266 | 281 | 140 |
| 1765856.66537 | 279 | 139 |
| 1770913.32237 | 277 | 138 |
| 1783868.94069 | 275 | 137 |
| 1816356.92480 | 273 | 136 |
| 1861068.90557 | 271 | 135 |
| 1897745.39786 | 269 | 134 |
| 1906333.59018 | 267 | 133 |
| 1941285.64389 | 265 | 132 |
| 1942048.29096 | 263 | 131 |
| 1962752.17933 | 261 | 130 |
| 1971926.79530 | 259 | 129 |
| 2100626.10051 | 257 | 128 |
| 2112010.95890 | 255 | 127 |
| 2137328.76712 | 253 | 126 |
| 2137512.66207 | 251 | 125 |
| 2140950.51809 | 249 | 124 |
| 2143348.36915 | 247 | 123 |
| 2146844.95434 | 245 | 122 |
| 2170708.42817 | 243 | 121 |
| 2206208.69336 | 241 | 120 |
| 2261200.55743 | 239 | 119 |
| 2279266.97295 | 237 | 118 |
| 2407196.80077 | 235 | 117 |
| 2440207.45978 | 233 | 116 |
| 2455322.21417 | 231 | 115 |
| 2455627.98547 | 227 | 113 |
| 2459377.47950 | 225 | 112 |
| 2464423.26882 | 223 | 111 |
| 2482236.95448 | 221 | 110 |
| 2545267.85094 | 217 | 108 |
| 2600620.24604 | 215 | 107 |
| 2603708.22627 | 213 | 106 |
| 2649456.13995 | 211 | 105 |
| 2675714.77264 | 209 | 104 |
| 2712033.68357 | 207 | 103 |
| 2742186.65221 | 205 | 102 |
| 2788576.72048 | 203 | 101 |

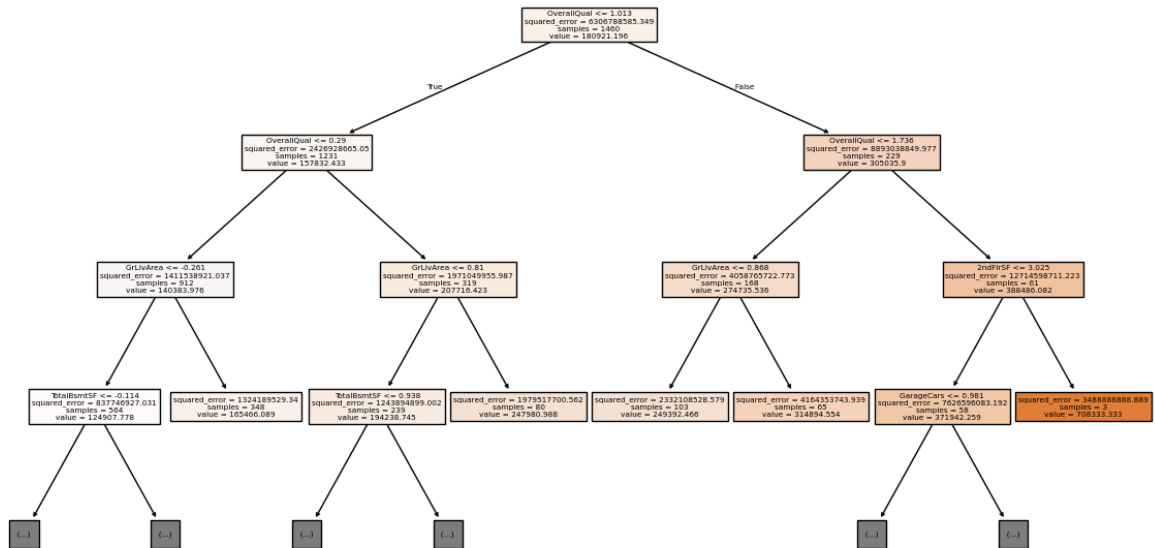
|                |     |     |
|----------------|-----|-----|
| 2821309.23649  | 201 | 100 |
| 2836010.10939  | 199 | 99  |
| 2841295.26422  | 197 | 98  |
| 2970616.81126  | 195 | 97  |
| 3068694.67466  | 193 | 96  |
| 3157636.19516  | 191 | 95  |
| 3375205.76484  | 189 | 94  |
| 3426478.25643  | 187 | 93  |
| 3450368.00551  | 185 | 92  |
| 3479835.46521  | 183 | 91  |
| 3523114.63710  | 181 | 90  |
| 3566083.07241  | 179 | 89  |
| 3587132.40510  | 177 | 88  |
| 3657484.93333  | 175 | 87  |
| 3695876.20267  | 173 | 86  |
| 3764042.04960  | 171 | 85  |
| 3859534.80697  | 169 | 84  |
| 3868448.57144  | 167 | 83  |
| 3935650.68493  | 165 | 82  |
| 3977291.03827  | 163 | 81  |
| 4062753.52768  | 161 | 80  |
| 4240989.83049  | 159 | 79  |
| 4443787.53619  | 157 | 78  |
| 4561672.51425  | 155 | 77  |
| 4572084.98706  | 151 | 75  |
| 4599576.20252  | 149 | 74  |
| 4620525.86712  | 147 | 73  |
| 4778108.45968  | 143 | 71  |
| 4876953.96096  | 141 | 70  |
| 4954632.68805  | 139 | 69  |
| 5145483.32022  | 137 | 68  |
| 5150567.05227  | 135 | 67  |
| 5206627.22104  | 133 | 66  |
| 5254025.82477  | 131 | 65  |
| 5318783.90138  | 129 | 64  |
| 5582371.87268  | 127 | 63  |
| 5663817.78528  | 125 | 62  |
| 6034885.97041  | 123 | 61  |
| 6075655.28358  | 121 | 60  |
| 6492363.23351  | 119 | 59  |
| 6510118.42112  | 117 | 58  |
| 6545510.15240  | 115 | 57  |
| 6733201.34276  | 113 | 56  |
| 7134703.19635  | 111 | 55  |
| 7203513.70975  | 109 | 54  |
| 7231840.38500  | 107 | 53  |
| 7810412.66002  | 105 | 52  |
| 8198687.21461  | 103 | 51  |
| 8256810.10272  | 101 | 50  |
| 8433074.30710  | 99  | 49  |
| 8892463.87850  | 97  | 48  |
| 9083254.47029  | 95  | 47  |
| 9322900.89538  | 93  | 46  |
| 9568005.20005  | 91  | 45  |
| 9871256.71737  | 89  | 44  |
| 10227222.25393 | 87  | 43  |

|                  |    |    |
|------------------|----|----|
| 10295471.66285   | 85 | 42 |
| 10535003.98022   | 83 | 41 |
| 10685352.74087   | 81 | 40 |
| 11753435.80563   | 79 | 39 |
| 11790105.32889   | 77 | 38 |
| 11875862.56794   | 75 | 37 |
| 12082259.42285   | 73 | 36 |
| 12157785.60329   | 71 | 35 |
| 13078117.81599   | 69 | 34 |
| 13981182.22990   | 65 | 32 |
| 14038751.27074   | 63 | 31 |
| 14711588.57352   | 61 | 30 |
| 15485120.01713   | 59 | 29 |
| 15525159.84942   | 57 | 28 |
| 18145687.19061   | 55 | 27 |
| 19209926.90266   | 53 | 26 |
| 19724750.88154   | 51 | 25 |
| 21134947.89214   | 49 | 24 |
| 21164078.01066   | 47 | 23 |
| 21561834.63664   | 45 | 22 |
| 22226343.41097   | 43 | 21 |
| 23773321.82235   | 41 | 20 |
| 27404346.64374   | 39 | 19 |
| 28654697.37451   | 37 | 18 |
| 32150052.03325   | 35 | 17 |
| 36597627.31580   | 33 | 16 |
| 40320920.20932   | 31 | 15 |
| 49007552.36479   | 29 | 14 |
| 50512459.27104   | 27 | 13 |
| 51366025.81746   | 25 | 12 |
| 52288095.15208   | 23 | 11 |
| 59221217.06456   | 21 | 10 |
| 60929792.73300   | 19 | 9  |
| 66432662.32815   | 17 | 8  |
| 93278712.48072   | 15 | 7  |
| 117111280.56582  | 13 | 6  |
| 118570300.72154  | 11 | 5  |
| 221083069.78959  | 9  | 4  |
| 242476899.26708  | 7  | 3  |
| 396604612.21529  | 5  | 2  |
| 733877229.26773  | 3  | 1  |
| 2865654966.63635 | 1  | 0  |

Chosen alpha: 59221217.06456

Final pruned tree: node\_count=21, n\_leaves=11, splits=10

Pruned CART Model (ccp\_alpha=59221217.06456)



`ccp_alpha` is the per-leaf penalty applied during pruning. Each potential split is weighed against how much it reduces impurity *minus* how much "cost" it adds (one more leaf). A larger `ccp_alpha` means each additional leaf must provide a large improvement in reducing impurity, so that the tree prunes aggressively. A smaller one means each leaf needs only a small improvement in impurity reduction to be "worth it", so the tree is allowed to grow deeper.

### What happens if we significantly increase `ccp_alpha` ?

The penalty for tree complexity goes up, so many splits are pruned away and you get a shallower or even very small tree.

### What if we significantly decrease `ccp_alpha` ?

The penalty goes down, so the tree is allowed to keep more splits and can become very large (potentially overfitting).

## Task 2.f

Check the value of the improvement in the first split:

- Explain the concept of improvement.
- Provide a numerical expression for how this improvement is computed for the first split.

When we split one node into two child nodes, we look at how much the sum of squared errors (SSE) decreases. In regression trees, the parent node has some Mean Squared Error  $MSE_1$  over  $K_1$  data points. Splitting it into two child nodes each has  $MSE_2$  and  $MSE_3$  over  $K_2$  and  $K_3$  data points, respectively.

For the first split (or any split in general), the improvement is calculated as were shown in lectures:

$$\text{Improvement} = \frac{(\text{MSE}_1 \times K_1) - (\text{MSE}_2 \times K_2 + \text{MSE}_3 \times K_3)}{\text{MSE}_1 \times K_1}$$

In other words, the improvement tells us how much relative error reduction we get from going from one region (node) to two regions (nodes).

```
In [9]: one_split_tree = DecisionTreeRegressor(max_depth=1, random_state=42)
one_split_tree.fit(X_preprocessed_df, y)
tree_ = one_split_tree.tree_

root_node = 0
left_node = tree_.children_left[root_node]
right_node = tree_.children_right[root_node]

MSE_1 = tree_.impurity[root_node]
K_1 = tree_.n_node_samples[root_node]
MSE_2 = tree_.impurity[left_node]
K_2 = tree_.n_node_samples[left_node]
MSE_3 = tree_.impurity[right_node]
K_3 = tree_.n_node_samples[right_node]

SSE_root = MSE_1 * K_1
SSE_children = (MSE_2 * K_2) + (MSE_3 * K_3)
improvement = (SSE_root - SSE_children) / SSE_root

print(f"MSE_1 (root): {MSE_1:.4f}, K_1: {K_1}")
print(f"MSE_2 (left): {MSE_2:.4f}, K_2: {K_2}")
print(f"MSE_3 (right): {MSE_3:.4f}, K_3: {K_3}")
print(f"Improvement for the first split = {improvement:.4%}")
```

```
MSE_1 (root): 6306788585.3493, K_1: 1460
MSE_2 (left): 2426928665.0497, K_2: 1231
MSE_3 (right): 8893038849.9768, K_3: 229
Improvement for the first split = 45.4376%
```

## Task 2.g

Compute the importance of the variables and compare the results to the importances from lasso regression (remaining variables) and linear regression.

```
In [16]: lin_coefs = linear_reg.coef_
ridge_coefs = ridge_cv.coef_
lasso_coefs = lasso_cv.coef_

# "Importance" as absolute value of each coefficient
lin_importance = np.abs(lin_coefs)
ridge_importance = np.abs(ridge_coefs)
lasso_importance = np.abs(lasso_coefs)
```

```

coef_df = pd.DataFrame({
    'Feature': feature_names,
    'Linear Coef': lin_coefs,
    'Linear Importance': lin_importance,
    'Ridge Coef': ridge_coefs,
    'Ridge Importance': ridge_importance,
    'Lasso Coef': lasso_coefs,
    'Lasso Importance': lasso_importance
})

n_lasso_zero = np.sum(lasso_coefs == 0.0)
n_lasso_nonzero = np.sum(lasso_coefs != 0.0)
print(f"Lasso zero-coeff features: {n_lasso_zero}")
print(f"Lasso non-zero features: {n_lasso_nonzero}")

top_k = 10 # top 10 by absolute magnitude

print("\nTop 10 (by absolute coefficient) - Linear Model:")
display(coef_df[['Feature', 'Linear Coef', 'Linear Importance']].
        sort_values('Linear Importance', ascending=False).head(top_k))

print("\nTop 10 (by absolute coefficient) - Ridge Model:")
display(coef_df[['Feature', 'Ridge Coef', 'Ridge Importance']].
        sort_values('Ridge Importance', ascending=False).head(top_k))

print("\nTop 10 (by absolute coefficient) - Lasso Model:")
display(coef_df[['Feature', 'Lasso Coef', 'Lasso Importance']].
        sort_values('Lasso Importance', ascending=False).head(top_k))

top_lin = (coef_df[['Feature', 'Linear Importance']].nlargest(top_k, 'Linear Importance'))
top_ridge = (coef_df[['Feature', 'Ridge Importance']].nlargest(top_k, 'Ridge Importance'))
top_lasso = (coef_df[['Feature', 'Lasso Importance']].nlargest(top_k, 'Lasso Importance'))

fig, axes = plt.subplots(1, 3, figsize=(18, 6))

axes[0].barh(top_lin['Feature'], top_lin['Linear Importance'], color='blue')
axes[0].invert_yaxis()
axes[0].set_title("Top 10 Linear Feature Importances")
axes[0].set_xlabel("Absolute Coefficient (Linear)")

axes[1].barh(top_ridge['Feature'], top_ridge['Ridge Importance'], color='green')
axes[1].invert_yaxis()
axes[1].set_title("Top 10 Ridge Feature Importances")
axes[1].set_xlabel("Absolute Coefficient (Ridge)")

axes[2].barh(top_lasso['Feature'], top_lasso['Lasso Importance'], color='red')
axes[2].invert_yaxis()
axes[2].set_title("Top 10 Lasso Feature Importances")
axes[2].set_xlabel("Absolute Coefficient (Lasso)")

plt.tight_layout()
plt.show()

```

Lasso zero-coeff features: 171  
Lasso non-zero features: 111

Top 10 (by absolute coefficient) - Linear Model:

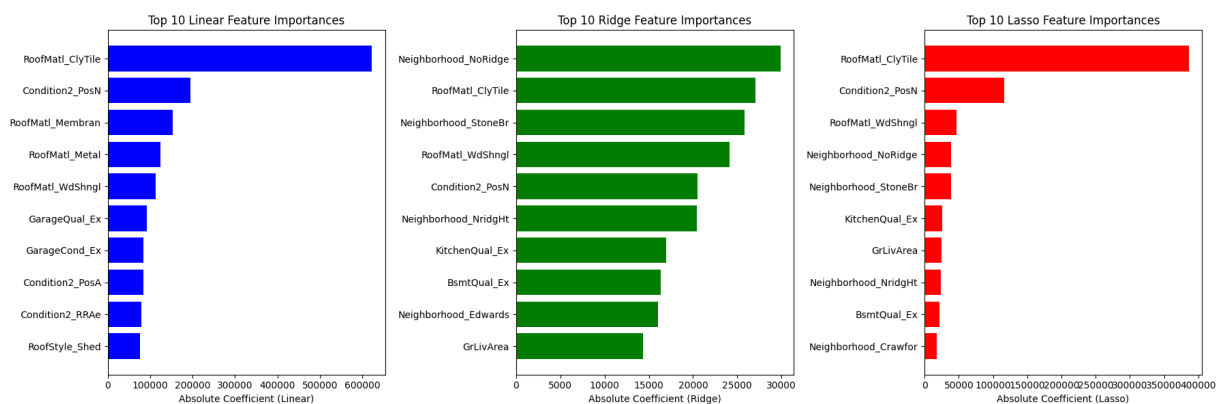
|            | Feature          | Linear Coef    | Linear Importance |
|------------|------------------|----------------|-------------------|
| <b>122</b> | RoofMatl_ClyTile | -622566.861332 | 622566.861332     |
| <b>99</b>  | Condition2_PosN  | -194332.220828 | 194332.220828     |
| <b>124</b> | RoofMatl_Membran | 153077.700553  | 153077.700553     |
| <b>125</b> | RoofMatl_Metal   | 124752.300998  | 124752.300998     |
| <b>129</b> | RoofMatl_WdShngl | 112901.515347  | 112901.515347     |
| <b>252</b> | GarageQual_Ex    | 91721.473350   | 91721.473350      |
| <b>258</b> | GarageCond_Ex    | -84124.844813  | 84124.844813      |
| <b>98</b>  | Condition2_PosA  | 83545.180240   | 83545.180240      |
| <b>100</b> | Condition2_RRAe  | -78945.073018  | 78945.073018      |
| <b>121</b> | RoofStyle_Shed   | 75206.131793   | 75206.131793      |

Top 10 (by absolute coefficient) - Ridge Model:

|            | Feature              | Ridge Coef    | Ridge Importance |
|------------|----------------------|---------------|------------------|
| <b>76</b>  | Neighborhood_NoRidge | 29871.671213  | 29871.671213     |
| <b>122</b> | RoofMatl_ClyTile     | -27053.896932 | 27053.896932     |
| <b>83</b>  | Neighborhood_StoneBr | 25840.810216  | 25840.810216     |
| <b>129</b> | RoofMatl_WdShngl     | 24135.759342  | 24135.759342     |
| <b>99</b>  | Condition2_PosN      | -20472.562057 | 20472.562057     |
| <b>77</b>  | Neighborhood_NridgHt | 20445.167596  | 20445.167596     |
| <b>224</b> | KitchenQual_Ex       | 16946.958108  | 16946.958108     |
| <b>176</b> | BsmtQual_Ex          | 16382.786692  | 16382.786692     |
| <b>68</b>  | Neighborhood_Edwards | -16024.203438 | 16024.203438     |
| <b>15</b>  | GrLivArea            | 14366.916134  | 14366.916134     |

Top 10 (by absolute coefficient) - Lasso Model:

|            | Feature              | Lasso Coef     | Lasso Importance |
|------------|----------------------|----------------|------------------|
| <b>122</b> | RoofMatl_ClyTile     | -385993.298958 | 385993.298958    |
| <b>99</b>  | Condition2_PosN      | -116020.356108 | 116020.356108    |
| <b>129</b> | RoofMatl_WdShngl     | 47108.629724   | 47108.629724     |
| <b>76</b>  | Neighborhood_NoRidge | 38944.244609   | 38944.244609     |
| <b>83</b>  | Neighborhood_StoneBr | 38602.001351   | 38602.001351     |
| <b>224</b> | KitchenQual_Ex       | 25356.057516   | 25356.057516     |
| <b>15</b>  | GrLivArea            | 24952.161443   | 24952.161443     |
| <b>77</b>  | Neighborhood_NridgHt | 24254.650139   | 24254.650139     |
| <b>176</b> | BsmtQual_Ex          | 21730.155996   | 21730.155996     |
| <b>67</b>  | Neighborhood_Crawfor | 18155.079867   | 18155.079867     |



Linear Regression picks up extreme values for some rare categories like roof materials and garage quality. These features have very large coefficients, which means the model is sensitive to small changes and could be unstable.

Ridge Regression spreads importance more evenly across features. It highlights neighborhoods and living area ( `GrLivArea` ) as key factors while keeping all features in the model.

Lasso Regression removes 171 features, leaving only 111 most important ones. It focuses on neighborhood, kitchen quality, and living area, being great for picking out the most important features and avoiding unnecessary noise.

## Task 2.h (optional)

Apply the bootstrap technique to estimate the distribution of the parameters in a linear regression:

- Consider the first 10 features only.

- For each step of the bootstrap, draw with replacement a sample of the same size as the original sample.
- Estimate the parameters by OLS and store the parameters for `LotArea` and `Street`.
- Repeat this 1000 times.
- Plot the histogram (or KDE) for these two parameters and compare them with the usual normal density (with mean  $\hat{\beta}$  and variance given by the diagonal element of  $\hat{\sigma}^2(X'X)^{-1}$ ).
- Is using the bootstrap a reasonable approach here?

```
In [25]: # Get the first 10 features and preprocess the data
desired_cols = [
    "LotArea", "Street", "OverallQual", "OverallCond",
    "YearBuilt", "YearRemodAdd", "TotalBsmtSF",
    "GrLivArea", "FullBath", "Fireplaces"
]

df_subset = df[desired_cols + ["SalePrice"]].copy()

y = df_subset["SalePrice"]
X = df_subset.drop(columns=["SalePrice"])

numeric_features = [
    "LotArea", "OverallQual", "OverallCond",
    "YearBuilt", "YearRemodAdd", "TotalBsmtSF",
    "GrLivArea", "FullBath", "Fireplaces"
]
categorical_features = ["Street"]

num_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

cat_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="constant", fill_value="Unknown")),
    ("encoder", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
])

preprocessor = ColumnTransformer([
    ("num", num_pipeline, numeric_features),
    ("cat", cat_pipeline, categorical_features)
])

X_preprocessed = preprocessor.fit_transform(X)
cat_encoder = preprocessor.named_transformers_["cat"].named_steps["encoder"]
cat_colnames = cat_encoder.get_feature_names_out(categorical_features)

feature_names = numeric_features + list(cat_colnames)

X_preprocessed_df = pd.DataFrame(X_preprocessed, columns=feature_names)
X_preprocessed_df.head()
```

|          | LotArea   | OverallQual | OverallCond | YearBuilt | YearRemodAdd | TotalBsmtS |
|----------|-----------|-------------|-------------|-----------|--------------|------------|
| <b>0</b> | -0.207142 | 0.651479    | -0.517200   | 1.050994  | 0.878668     | -0.45930   |
| <b>1</b> | -0.091886 | -0.071836   | 2.179628    | 0.156734  | -0.429577    | 0.46646    |
| <b>2</b> | 0.073480  | 0.651479    | -0.517200   | 0.984752  | 0.830215     | -0.31336   |
| <b>3</b> | -0.096897 | 0.651479    | -0.517200   | -1.863632 | -0.720298    | -0.68732   |
| <b>4</b> | 0.375148  | 1.374795    | -0.517200   | 0.951632  | 0.733308     | 0.19968    |

```

In [28]: # OLS regression
X_sm = sm.add_constant(X_preprocessed_df)

model_full = sm.OLS(y, X_sm).fit()
print(model_full.summary())

print("\nCoefficients:\n", model_full.params)

```

# OLS Regression Results

```

=====
==
Dep. Variable:          SalePrice    R-squared:                0.7
76
Model:                  OLS          Adj. R-squared:           0.7
75
Method:                 Least Squares    F-statistic:              50
3.2
Date:                   Fri, 31 Jan 2025    Prob (F-statistic):       0.
00
Time:                   00:16:42          Log-Likelihood:           -1745
1.
No. Observations:      1460             AIC:                      3.492e+
04
Df Residuals:          1449             BIC:                      3.498e+
04
Df Model:               10
Covariance Type:       nonrobust
=====

```

```

=====
=====
              coef      std err          t      P>|t|      [0.025      0.
975]
-----
----
const      1.161e+05    5280.211     21.995     0.000     1.06e+05     1.26
e+05
LotArea    6265.1639     1088.094      5.758     0.000     4130.756     839
9.572
OverallQual 2.759e+04     1637.987     16.844     0.000     2.44e+04     3.08
e+04
OverallCond 5719.6111     1184.861      4.827     0.000     3395.385     804
3.837
YearBuilt   1.502e+04     1638.455      9.168     0.000     1.18e+04     1.82
e+04
YearRemodAdd 4370.5617     1421.457      3.075     0.002     1582.228     715
8.895
TotalBsmtSF 1.234e+04     1257.425      9.813     0.000     9872.293     1.48
e+04
GrLivArea   2.761e+04     1607.366     17.179     0.000     2.45e+04     3.08
e+04
FullBath    -2050.7088     1456.549     -1.408     0.159    -4907.880      80
6.462
Fireplaces   5426.7668     1168.041      4.646     0.000     3135.535     771
7.999
Street_Grvl   5.13e+04     1.05e+04      4.864     0.000     3.06e+04     7.2
e+04
Street_Pave  6.484e+04     5314.404     12.200     0.000     5.44e+04     7.53
e+04
=====

```

```

==
Omnibus:              590.625    Durbin-Watson:              1.9
85
Prob(Omnibus):         0.000    Jarque-Bera (JB):           93550.4
86
Skew:                  -0.776    Prob(JB):                   0.

```

00  
Kurtosis: 42.184 Cond. No. 5.04e+  
16

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.  
[2] The smallest eigenvalue is 2.08e-30. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

Coefficients:

|              |               |
|--------------|---------------|
| const        | 116140.108496 |
| LotArea      | 6265.163850   |
| OverallQual  | 27590.196553  |
| OverallCond  | 5719.611110   |
| YearBuilt    | 15020.982227  |
| YearRemodAdd | 4370.561735   |
| TotalBsmtSF  | 12338.861674  |
| GrLivArea    | 27612.153930  |
| FullBath     | -2050.708761  |
| Fireplaces   | 5426.766751   |
| Street_Grvl  | 51303.404804  |
| Street_Pave  | 64836.703692  |

dtype: float64

```
In [29]: # OLS estimates and standard errors for columns of interest
lotarea_hat = model_full.params["LotArea"]
lotarea_se = model_full.bse["LotArea"]

street_grvl_hat = model_full.params.get("Street_Grvl", np.nan)
street_grvl_se = model_full.bse.get("Street_Grvl", np.nan)
street_pave_hat = model_full.params.get("Street_Pave", np.nan)
street_pave_se = model_full.bse.get("Street_Pave", np.nan)
```

```
In [30]: # Bootstrap 1000x, resample rows with replacement, refit OLS
n = len(X_sm)
n_boot = 1000

boot_lotarea = []
boot_street_grvl = []
boot_street_pave = []

for _ in range(n_boot):
    indices = np.random.randint(0, n, size=n)
    Xb = X_sm.iloc[indices, :]
    yb = y.iloc[indices]

    model_b = sm.OLS(yb, Xb).fit()

    boot_lotarea.append(model_b.params.get("LotArea", np.nan))
    boot_street_grvl.append(model_b.params.get("Street_Grvl", np.nan))
    boot_street_pave.append(model_b.params.get("Street_Pave", np.nan))

boot_lotarea = np.array(boot_lotarea)
```

```

boot_street_grvl = np.array(boot_street_grvl)
boot_street_pave = np.array(boot_street_pave)

# Histograms/KDE vs Normal Approx
fig, axes = plt.subplots(1, 3, figsize=(18,5), sharey=False)

ax = axes[0]
sns.histplot(boot_lotarea, kde=True, color="skyblue", stat="density", ax=ax)
ax.set_title("Bootstrap Dist: LotArea")
ax.set_xlabel("Coefficient Estimate")

xvals = np.linspace(boot_lotarea.min(), boot_lotarea.max(), 200)
pdf_vals = stats.norm.pdf(xvals, loc=lotarea_hat, scale=lotarea_se)
ax.plot(xvals, pdf_vals, 'r--', label='Normal approx')
ax.legend()

ax_grvl = axes[1]
sns.histplot(boot_street_grvl, kde=True, color="orange", stat="density", ax=ax_grvl)
ax_grvl.set_title("Bootstrap Dist: Street_Grvl")
ax_grvl.set_xlabel("Coefficient Estimate")

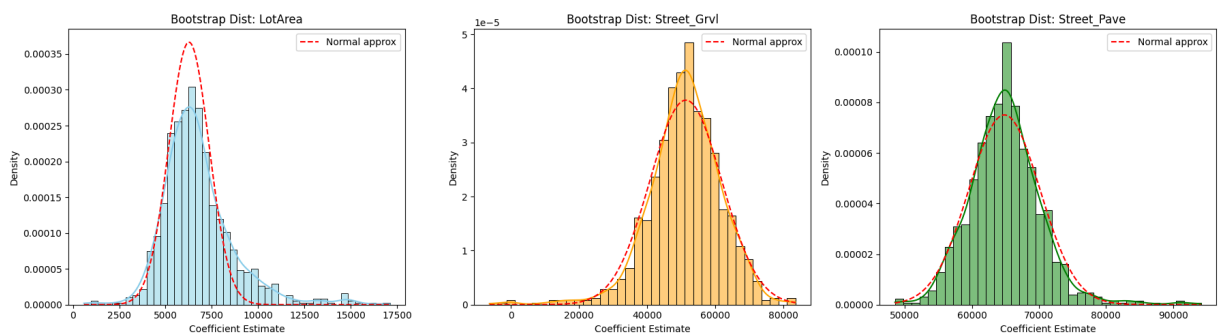
xvals_grvl = np.linspace(boot_street_grvl.min(), boot_street_grvl.max(), 200)
pdf_vals_grvl = stats.norm.pdf(xvals_grvl, loc=street_grvl_hat, scale=street_grvl_se)
ax_grvl.plot(xvals_grvl, pdf_vals_grvl, 'r--', label='Normal approx')
ax_grvl.legend()

ax_pave = axes[2]
sns.histplot(boot_street_pave, kde=True, color="green", stat="density", ax=ax_pave)
ax_pave.set_title("Bootstrap Dist: Street_Pave")
ax_pave.set_xlabel("Coefficient Estimate")

xvals_pave = np.linspace(boot_street_pave.min(), boot_street_pave.max(), 200)
pdf_vals_pave = stats.norm.pdf(xvals_pave, loc=street_pave_hat, scale=street_pave_se)
ax_pave.plot(xvals_pave, pdf_vals_pave, 'r--', label='Normal approx')
ax_pave.legend()

plt.tight_layout()
plt.show()

```



**Is using the bootstrap a reasonable approach here?**

In standard OLS regression, we usually assume that errors are independent, identically distributed (i.i.d.), and have constant variance. Under these

conditions, the normal approximation for coefficient estimates works well, especially when the dataset is large.

However, the bootstrap can be helpful when the dataset is small (like in our case with 10 features only), making the usual normal approximation less reliable. Also if there are violations of standard OLS assumptions, such as outliers or heteroskedasticity (unequal variance) or if we want an empirical check on the distribution of our coefficient estimates rather than relying purely on theory.

In most well-behaved regression settings with enough data, the bootstrap and the classical normal approximation often give similar results. But it is still a reasonable tool, particularly for validating or double-checking the theoretical standard errors from OLS.

## Task 2.i (optional)

Having the sample of the parameters:

- Compute the confidence interval defined by the 2.5% and 97.5% quantiles of the sample.
- Compare these to the confidence intervals based on the assumption of normality.

```
In [32]: # Empirical bootstrap 95% CI (2.5%, 97.5% quantiles)
lotarea_ci_boot = np.percentile(boot_lotarea, [2.5, 97.5])

# Normal-based 95% CI (assuming large-sample normality)
z_val = 1.96 # ~95%
lotarea_ci_normal = [
    lotarea_hat - z_val * lotarea_se,
    lotarea_hat + z_val * lotarea_se
]

print(f"LotArea parameter estimates:")
print(f"Bootstrap 95% CI: [{lotarea_ci_boot[0]:.4f}, {lotarea_ci_boot[1]:.4f}]")
print(f"Normal-based 95% CI: [{lotarea_ci_normal[0]:.4f}, {lotarea_ci_normal[1]:.4f}]")

street_grvl_hat = model_full.params["Street_Grvl"]
street_grvl_se = model_full.bse["Street_Grvl"]

street_grvl_ci_boot = np.percentile(boot_street_grvl, [2.5, 97.5])
street_grvl_ci_normal = [
    street_grvl_hat - z_val * street_grvl_se,
    street_grvl_hat + z_val * street_grvl_se
]

print(f"\nStreet_Grvl parameter estimates:")
print(f"Bootstrap 95% CI: [{street_grvl_ci_boot[0]:.4f}, {street_grvl_ci_boot[1]:.4f}]")
print(f"Normal-based 95% CI: [{street_grvl_ci_normal[0]:.4f}, {street_grvl_ci_normal[1]:.4f}]")
```

```

street_pave_hat = model_full.params["Street_Pave"]
street_pave_se = model_full.bse["Street_Pave"]

street_pave_ci_boot = np.percentile(boot_street_pave, [2.5, 97.5])
street_pave_ci_normal = [
    street_pave_hat - z_val * street_pave_se,
    street_pave_hat + z_val * street_pave_se
]

print(f"\nStreet_Pave parameter estimates:")
print(f"Bootstrap 95% CI: [{street_pave_ci_boot[0]:.4f}, {street_pave_ci_boot[1]:.4f}]")
print(f"Normal-based 95% CI: [{street_pave_ci_normal[0]:.4f}, {street_pave_ci_normal[1]:.4f}]")

```

LotArea parameter estimates:  
 Bootstrap 95% CI: [4013.2337, 11661.8251]  
 Normal-based 95% CI: [4132.4998, 8397.8279]

Street\_Grvl parameter estimates:  
 Bootstrap 95% CI: [28670.8243, 69724.4790]  
 Normal-based 95% CI: [30628.4579, 71978.3517]

Street\_Pave parameter estimates:  
 Bootstrap 95% CI: [55631.4544, 76457.7054]  
 Normal-based 95% CI: [54420.4713, 75252.9361]

For `LotArea`, the bootstrap interval is noticeably wider than the normal-based range, so that when we rely on empirical resampling rather than the standard OLS formula, there's more uncertainty in the upper bound than the classical approach implies.

For `Street_Grvl`, the two methods produce fairly similar results: the intervals differ slightly on the lower end, but overall they tell the same story. The difference here isn't significant enough to suggest major concerns about the standard normal approximation.

With `Street_Pave`, the two intervals are nearly identical, and this consistency implies that both approaches agree on the level of uncertainty for this coefficient.

Overall, the bootstrap tends to produce slightly wider or shifted confidence intervals, especially when the data deviates from normality or has some skewness. However, in this case, most of the intervals are quite close to each other, indicating that there are no major violations of the usual OLS assumptions.

## Task 2.j

Use bagging and random forest to predict `SalePrice`. Plot the error as a function of the number of trees.

```

In [38]: # Split the data into train / test samples
X_train, X_test, y_train, y_test = train_test_split(
    X_preprocessed, y,
    test_size=0.2,
    random_state=42
)

n_estimators_list = [1, 5, 10, 50, 100, 200, 300]

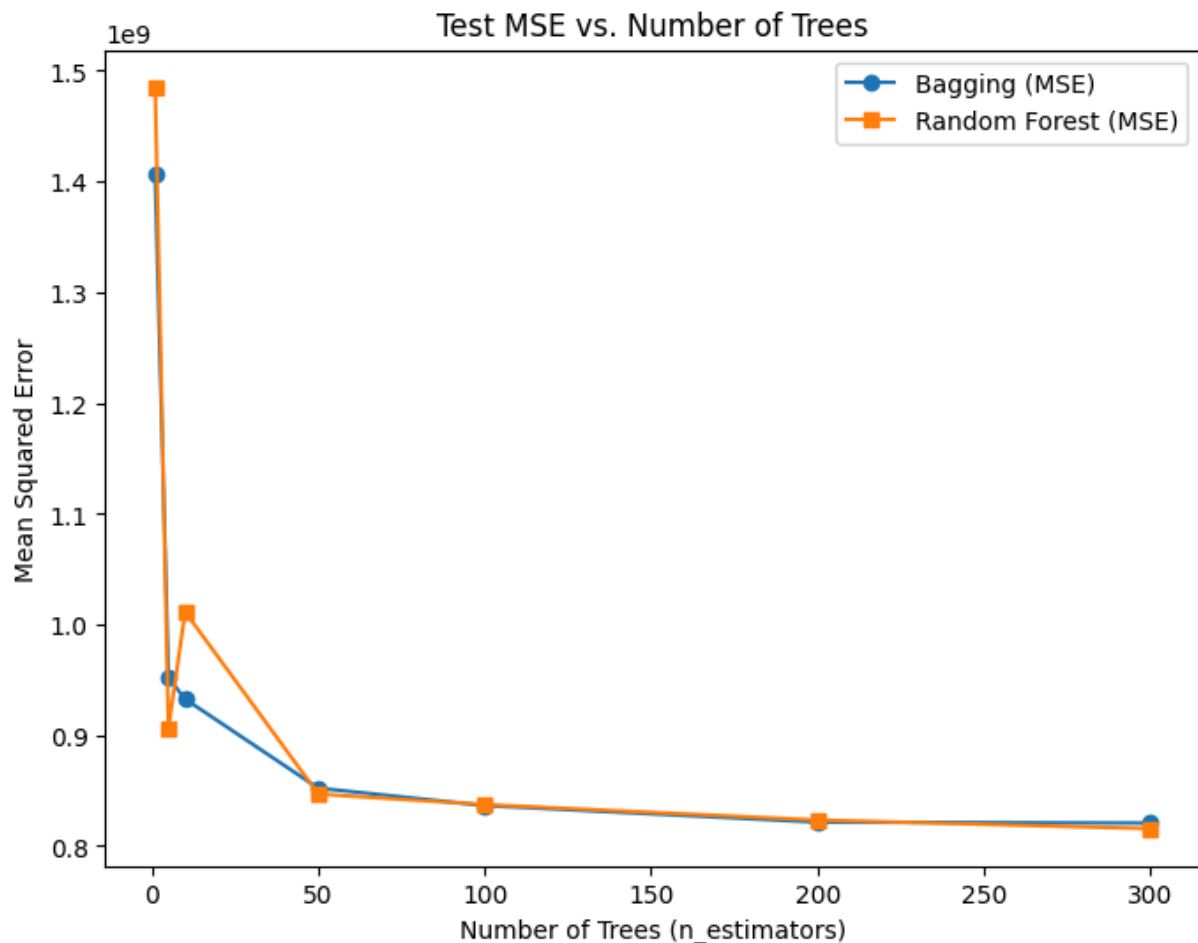
bagging_test_mse = []
rf_test_mse = []

for n in n_estimators_list:
    # Bagging
    bag_model = BaggingRegressor(
        n_estimators=n,
        random_state=42
    )
    bag_model.fit(X_train, y_train)
    y_pred_bag = bag_model.predict(X_test)
    mse_bag = mean_squared_error(y_test, y_pred_bag)
    bagging_test_mse.append(mse_bag)

    # Random Forest
    rf_model = RandomForestRegressor(
        n_estimators=n,
        random_state=42
    )
    rf_model.fit(X_train, y_train)
    y_pred_rf = rf_model.predict(X_test)
    mse_rf = mean_squared_error(y_test, y_pred_rf)
    rf_test_mse.append(mse_rf)

# Error vs. Number of Trees
plt.figure(figsize=(8, 6))
plt.plot(n_estimators_list, bagging_test_mse, marker='o', label="Bagging (MSE)")
plt.plot(n_estimators_list, rf_test_mse, marker='s', label="Random Forest (MSE)")
plt.title("Test MSE vs. Number of Trees")
plt.xlabel("Number of Trees (n_estimators)")
plt.ylabel("Mean Squared Error")
plt.legend()
plt.show()

```



In Bagging we use many random subsamples (with replacement) to train multiple base estimators (by default, `DecisionTreeRegressor`). Predictions are averaged across all estimators, which reduces variance compared to a single tree. Random Forest also uses many decision trees but additionally randomly selects subsets of features at each split. Typically converges faster and to a lower error than plain bagging with the same number of trees.

As `n_estimators` increases, both methods decrease their test error and eventually plateau, indicating that more trees help stabilize the ensemble.

## Task 2.k

Check the computational times for both approaches:

- Why is random forest typically faster and requires fewer trees?
- Check which approach is implemented in your software for variable importance.
- Compute these and compare them to the results for lasso, a single tree, and linear regression.

```

In [41]: models = {
    "LinearRegression": LinearRegression(),
    "Lasso": Lasso(alpha=1.0, max_iter=10000, random_state=42),
    "SingleTree": DecisionTreeRegressor(random_state=42),
    "Bagging": BaggingRegressor(random_state=42, n_estimators=200),
    "RandomForest": RandomForestRegressor(random_state=42, n_estimators=200)
}

train_times = {}
test_mse = {}

for name, model in models.items():
    start = time.time()
    model.fit(X_train, y_train)
    end = time.time()

    train_times[name] = end - start

    y_pred = model.predict(X_test)
    test_mse[name] = mean_squared_error(y_test, y_pred)

print("Training time (seconds):")
for k, v in train_times.items():
    print(f" {k:<15} : {v:.4f}")

print("\nTest MSE:")
for k, v in test_mse.items():
    print(f" {k:<15} : {v:.2f}")

```

```

Training time (seconds):
LinearRegression : 0.0461
Lasso           : 0.6974
SingleTree      : 0.0820
Bagging         : 8.3792
RandomForest    : 8.2666

```

```

Test MSE:
LinearRegression : 882259847.83
Lasso           : 800702980.69
SingleTree      : 2056022514.22
Bagging         : 821477257.55
RandomForest    : 823337541.71

```

### Why is Random Forest faster and needs fewer trees?

Random Forest speeds up training because each tree uses a random subset of features at each split, reducing the number of calculations. This also helps decorrelate trees, so we often need fewer estimators to get strong results.

In contrast, Bagging with Decision Trees uses all features at every split unless we manually limit them. This can lead to more correlated trees and requires more estimators to reduce variance, making training slower.

### Which model for variable importance?

- Random Forest has built-in `feature_importances_`, which measures how much each feature reduces impurity (Gini importance), or permutation importance can also be used.
- Bagging doesn't directly provide feature importance unless extracted and averaged from individual trees.
- Decision Tree also has `feature_importances_`, but it only applies to a single tree. In other cases can show which features it used for splits.
- Lasso & Linear Regression have no built-in feature importance, but we can use the absolute value of coefficients as an example.

## Comparing models

### 1. Training Speed:

- Fastest: Linear Regression, Lasso
- Medium: Single Decision Tree
- Slowest: Bagging, Random Forest (but Random Forest converges faster than plain bagging)

### 2. Test Performance (MSE):

- Best: Random Forest (handles non-linearities well)
- Good: Lasso (works well when many features are irrelevant)
- Decent: Single Tree (tend to overfitting)
- Worst: Plain Linear Regression (if relationships are non-linear)

If we want *fast and simple interpretability*, go for *Lasso*. If we need *high accuracy on tabular data*, *Random Forest* is a strong choice. *Bagging* can work too, but *Random Forest* is often the *better version* with smarter feature selection.

If we're looking for *more challenges and frustration in life*, we can start with *Linear Regression*, struggle to make it work, and then desperately turn to *Non-Linear Regression* and various transformations. Eventually, we'll throw all the data into an *LSTM* or *some other neural network*, watch it magically work, and pat ourselves on the back for being brilliant ML engineers.

# Task 3 Classifying houses based on Sale Price

Next, we classify the houses by defining:

$$Y_i = \begin{cases} 1, & \text{if SalePrice} > 130000 \\ 0, & \text{otherwise} \end{cases}$$

The aim is to classify houses into these two classes.

## Task 3.a

Estimate the logistic regression using a backward model selection with AIC as the selection criterion. To speed up computation, consider only numeric features in the initial model. Further, work only with the final model.

```
In [42]: # Data preprocessing
df["TargetClass"] = (df["SalePrice"] > 130000).astype(int)

threshold = 0.5 * len(df)
df = df.dropna(thresh=threshold, axis=1)

y = df["TargetClass"]

# Keep only numeric columns
candidate_numeric_cols = df.select_dtypes(include=["int64", "float64"]).columns
candidate_numeric_cols = [col for col in candidate_numeric_cols
                           if col not in ["Id", "SalePrice", "TargetClass"]]

X_num = df[candidate_numeric_cols].copy()

# Fill numeric missing with median
X_num = X_num.fillna(X_num.median())

X_num.head()
```

```
Out[42]:
```

|   | MSSubClass | LotFrontage | LotArea | OverallQual | OverallCond | YearBuilt | YearRemodded |
|---|------------|-------------|---------|-------------|-------------|-----------|--------------|
| 0 | 60         | 65.0        | 8450    | 7           | 5           | 2003      | 2003         |
| 1 | 20         | 80.0        | 9600    | 6           | 8           | 1976      | 1976         |
| 2 | 60         | 68.0        | 11250   | 7           | 5           | 2001      | 2001         |
| 3 | 70         | 60.0        | 9550    | 7           | 5           | 1915      | 1915         |
| 4 | 60         | 84.0        | 14260   | 8           | 5           | 2000      | 2000         |

5 rows × 36 columns

At this point we will get **LinAlgError("Singular matrix")** indicating that during the logistic regression fit, the design matrix became rank-deficient. In other words, there is perfect (or near-perfect) multicollinearity among our features, preventing the maximum likelihood estimation from converging. So let's add some more preprocessing techniques.

```
In [87]: # Drop zero-variance
zero_var_cols = [c for c in X_num.columns if X_num[c].nunique() <= 1]
if zero_var_cols:
    print("Dropping zero-variance cols:", zero_var_cols)
X_num2 = X_num.drop(columns=zero_var_cols)

# Remove highly correlated columns
corr_matrix = X_num2.corr().abs()
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
high_corr_cols = [col for col in upper.columns if any(upper[col] > 0.9)]
if high_corr_cols:
    print("Dropping highly correlated cols:", high_corr_cols)
X_num3 = X_num2.drop(columns=high_corr_cols)

# Scale the numeric data
scaler = StandardScaler()
X_scaled_array = scaler.fit_transform(X_num3)
X_scaled = pd.DataFrame(X_scaled_array, columns=X_num3.columns)
```

Dropping highly correlated cols: ['1stFlrSF', 'GrLivArea', 'BsmtFullBath', 'FullBath', 'HalfBath', 'TotRmsAbvGrd', 'GarageYrBlt', 'GarageCars', 'GarageArea']

```
In [79]: def backward_elimination_aic_regularized(X, y, alpha=0.001, method='l1', verbose=False):
    """
    Perform backward elimination (one variable at a time) to minimize AIC
    in a logistic regression that uses fit_regularized to avoid singularities
    """
    included = list(X.columns)

    while True:
        # Fit model with current set of columns using fit_regularized
        X_sm = sm.add_constant(X[included], prepend=False)
        try:
            model = sm.Logit(y, X_sm).fit_regularized(
                method=method, alpha=alpha, maxiter=20000, disp=False
            )
        except (LinAlgError, ValueError) as e:
            if verbose:
                print("Fit failed with columns:", included, "Error:", e)
            break

        current_aic = model.aic
        best_aic = current_aic
        worst_feature = None

        # Try removing each feature in turn
        for col in included:
```

```

cols_minus_one = [f for f in included if f != col]
X_sm_minus = sm.add_constant(X[cols_minus_one], prepend=False)
try:
    model_minus = sm.Logit(y, X_sm_minus).fit_regularized(
        method=method, alpha=alpha, maxiter=20000, disp=False
    )
    new_aic = model_minus.aic

    if new_aic < best_aic:
        best_aic = new_aic
        worst_feature = col
except (LinAlgError, ValueError):
    # If dropping caused a failure, skip it
    continue

if worst_feature is not None:
    included.remove(worst_feature)
    if verbose:
        print(f"Dropped {worst_feature}, AIC improved from {current_
else:
    break

# Final model with the chosen set
final_X_sm = sm.add_constant(X[included], prepend=False)
try:
    final_model = sm.Logit(y, final_X_sm).fit_regularized(
        method=method, alpha=alpha, maxiter=20000, disp=False
    )
except Exception as e:
    if verbose:
        print("Final model fit failed:", e)
    final_model = None

if final_model is not None and verbose:
    print("\n=====")
    print(f"Selected features: {included}")
    print(f"Final AIC: {final_model.aic:.2f}")
    print("=====\n")

return included, final_model

```

```

In [88]: selected_cols, final_model = backward_elimination_aic_regularized(X_scaled,
print(final_model.summary())

X_final = sm.add_constant(X_num[selected_cols])
y_pred_prob = final_model.predict(X_final)
y_pred_class = (y_pred_prob >= 0.5).astype(int)

accuracy = accuracy_score(y, y_pred_class)
print("In-sample accuracy:", accuracy)

```

```
/usr/local/lib/python3.11/dist-packages/statsmodels/base/l1_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 3 out of 28 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
    warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/l1_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
    warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/l1_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 3 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
    warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/l1_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
    warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/l1_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 4 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
    warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/l1_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
    warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/l1_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 1 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
    warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/l1_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
    warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/l1_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 3 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
    warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/l1_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
    warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/l1_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 1 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
    warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/l1_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
    warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/l1_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 3 out of 27 parameters
```

Try increasing solver accuracy or number of iterations, decreasing alpha, or switch solvers

```
warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
```

```
warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 3 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
```

```
warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
```

```
warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 3 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
```

```
warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
```

```
warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 3 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
```

```
warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
```

```
warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 3 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
```

```
warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
```

```
warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 3 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
```

```
warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
```

```
warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 3 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
```

```
warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
    warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 4 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
    warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
    warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 1 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
    warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
    warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 3 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
    warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
    warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 3 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
    warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
    warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 3 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
    warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
    warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 3 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
    warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
```

```
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
    warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 2 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
    warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
    warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 3 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
    warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
    warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 3 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
    warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
    warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 3 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
    warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
    warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 3 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
    warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
    warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 3 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
    warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
```

```
warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 3 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
    warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
    warnings.warn(msg, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:71: ConvergenceWarning: QC check did not pass for 3 out of 27 parameters
Try increasing solver accuracy or number of iterations, decreasing alpha, or
switch solvers
    warnings.warn(message, ConvergenceWarning)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/ll_solvers_common.p
y:144: ConvergenceWarning: Could not trim params automatically due to failed
QC check. Trimming using trim_mode == 'size' will still work.
    warnings.warn(msg, ConvergenceWarning)
```

Dropped PoolArea, AIC improved from 656.70 to 650.78  
Dropped EnclosedPorch, AIC improved from 650.78 to 647.88  
Dropped MasVnrArea, AIC improved from 647.88 to 645.90  
Dropped MSSubClass, AIC improved from 645.90 to 643.97  
Dropped BsmtHalfBath, AIC improved from 643.97 to 642.04  
Dropped LowQualFinSF, AIC improved from 642.04 to 640.73  
Dropped MoSold, AIC improved from 640.73 to 639.84  
Dropped KitchenAbvGr, AIC improved from 639.84 to 639.00  
Dropped BedroomAbvGr, AIC improved from 639.00 to 638.47  
Dropped YrSold, AIC improved from 638.47 to 638.21  
Dropped 3SsnPorch, AIC improved from 638.21 to 638.07  
Dropped OpenPorchSF, AIC improved from 638.07 to 638.07  
Dropped MiscVal, AIC improved from 638.07 to 638.07  
Dropped BsmtUnfSF, AIC improved from 638.07 to 638.07  
Dropped BsmtFinSF2, AIC improved from 638.07 to 636.43

=====

Selected features: ['LotFrontage', 'LotArea', 'OverallQual', 'OverallCond',  
'YearBuilt', 'YearRemodAdd', 'BsmtFinSF1', 'TotalBsmtSF', '2ndFlrSF', 'Fireplaces',  
'WoodDeckSF', 'ScreenPorch']

Final AIC: 636.43

=====

#### Logit Regression Results

=====

==

|                  |                  |                   |          |
|------------------|------------------|-------------------|----------|
| Dep. Variable:   | TargetClass      | No. Observations: | 14       |
| 60               |                  |                   |          |
| Model:           | Logit            | Df Residuals:     | 14       |
| 47               |                  |                   |          |
| Method:          | MLE              | Df Model:         |          |
| 12               |                  |                   |          |
| Date:            | Fri, 31 Jan 2025 | Pseudo R-squ.:    | 0.63     |
| 35               |                  |                   |          |
| Time:            | 02:01:25         | Log-Likelihood:   | -305.    |
| 22               |                  |                   |          |
| converged:       | True             | LL-Null:          | -832.    |
| 87               |                  |                   |          |
| Covariance Type: | nonrobust        | LLR p-value:      | 2.387e-2 |
| 18               |                  |                   |          |

=====

=====

|             | coef   | std err | z     | P> z  | [0.025 | 0. |
|-------------|--------|---------|-------|-------|--------|----|
| 975]        |        |         |       |       |        |    |
| -----       |        |         |       |       |        |    |
| -----       |        |         |       |       |        |    |
| LotFrontage | 0.4496 | 0.175   | 2.573 | 0.010 | 0.107  |    |
| 0.792       |        |         |       |       |        |    |
| LotArea     | 2.4387 | 0.441   | 5.528 | 0.000 | 1.574  |    |
| 3.303       |        |         |       |       |        |    |
| OverallQual | 1.2044 | 0.202   | 5.968 | 0.000 | 0.809  |    |
| 1.600       |        |         |       |       |        |    |
| OverallCond | 0.5936 | 0.128   | 4.646 | 0.000 | 0.343  |    |
| 0.844       |        |         |       |       |        |    |
| YearBuilt   | 1.4551 | 0.177   | 8.201 | 0.000 | 1.107  |    |
| 1.803       |        |         |       |       |        |    |

|              |        |       |        |       |       |
|--------------|--------|-------|--------|-------|-------|
| YearRemodAdd | 0.4779 | 0.126 | 3.797  | 0.000 | 0.231 |
| 0.725        |        |       |        |       |       |
| BsmtFinSF1   | 0.6629 | 0.167 | 3.981  | 0.000 | 0.336 |
| 0.989        |        |       |        |       |       |
| TotalBsmtSF  | 1.3123 | 0.201 | 6.535  | 0.000 | 0.919 |
| 1.706        |        |       |        |       |       |
| 2ndFlrSF     | 1.6404 | 0.205 | 8.017  | 0.000 | 1.239 |
| 2.041        |        |       |        |       |       |
| Fireplaces   | 0.6720 | 0.136 | 4.930  | 0.000 | 0.405 |
| 0.939        |        |       |        |       |       |
| WoodDeckSF   | 0.2817 | 0.127 | 2.221  | 0.026 | 0.033 |
| 0.530        |        |       |        |       |       |
| ScreenPorch  | 0.3614 | 0.168 | 2.153  | 0.031 | 0.032 |
| 0.690        |        |       |        |       |       |
| const        | 3.8497 | 0.256 | 15.029 | 0.000 | 3.348 |
| 4.352        |        |       |        |       |       |

=====

=====

Possibly complete quasi-separation: A fraction 0.15 of observations can be perfectly predicted. This might indicate that there is complete quasi-separation. In this case some parameters will not be identified.  
In-sample accuracy: 0.7424657534246575

## Task 3.b

Consider the explanatory variable `LotArea`. Its parameter cannot be interpreted in the same way as in a linear regression. Provide the correct interpretation using the estimated parameter and odds.

For the final set of features  $\mathbf{x}$ , the model has the form:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_{\text{LotFrontage}} \cdot x_{\text{LotFrontage}} + \dots + \beta_{\text{ScreenPorch}} \cdot x_{\text{ScreenPorch}}$$

where  $p = P(\text{SalePrice} > 130k)$ .

A 1-unit increase in a coefficient's variable increases the log-odds by  $\beta$ . Equivalently, it multiplies the odds by  $\exp(\beta)$ .

`LotArea` coefficient is 2.4387. For a 1-unit increase in `LotArea` (here 1 standard deviation because the data is scaled), the log-odds goes up by 2.4387 and the odds multiply by  $\exp(2.4387) \approx 11.46$ . In other words, a 1-SD increase in `LotArea` is associated with about an 11-fold increase in the odds of having `SalePrice > 130000`.

Similarly `OverallQual` has a coefficient of about 1.2044 => a 1-SD increase multiplies the odds by  $\exp(1.2044) \approx 3.33$ . `2ndFlrSF` has 1.6404 =>  $\exp(1.6404) \approx 5.15$  times the odds for each standard deviation increase, etc.

All the remaining variables have positive coefficients too, indicating that larger values of these features make it more likely the house's sale price exceeds \$130k.

## Task 3.c

Select several variables that increase the probability of  $Y = 1$  and several variables that decrease this probability. Is this consistent with economic intuition?

From the final logistic regression model, *all selected features have positive coefficients*, meaning they *increase* the probability of a house selling above \$130K.

- Bigger lot and home size → LotArea , 2ndFlrSF , TotalBsmtSF
- Better home quality and condition → OverallQual , OverallCond
- Newer homes → YearBuilt , YearRemodAdd
- Extra features → Fireplaces , WoodDeckSF , ScreenPorch

Since *no variable has a negative coefficient*, the model suggests that all retained factors contribute positively to home value.

**Does this make economic sense?** Yes. Larger, newer, higher-quality homes with more features typically sell for more, which aligns with real estate market logic.

**Why no negative effects?** Backward selection likely dropped any variables that weren't statistically useful. If coded differently, some features (for instance, home age) *could* have a negative impact, but they didn't make the final cut.

## Task 3.d

- Randomly pick five houses.
- Determine their probabilities of having a price above 130000.
- Provide the formula for the probability of the first house, inserting the values of the parameters and variables.
- If you want to predict the membership in one of the two groups for a particular house, what is the simplest way to proceed using these probabilities?

The final model:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 \cdot x_1 + \beta_2 \cdot x_2 + \cdots + \beta_k \cdot x_k,$$

where  $p = P(\text{SalePrice} > 130000)$ . We have  $K$  features in the final model (plus an intercept).

The final logistic regression gave us the intercept  $\beta_0 = 3.8497$  and the other  $K = 12$  coefficients (for example for `LotArea = 2.4387`), so we can just plug them in the specific house's feature values (in scaled units) into:

$$\hat{p} = \frac{1}{1 + \exp[-(\beta_0 + \beta_1 x_1 + \dots + \beta_{12} x_{12})]} = \frac{1}{1 + \exp[-z_A]},$$

where  $z_A = \beta_0 + \beta_1 x_1 + \dots + \beta_{12} x_{12}$ . This would give us each of the five probabilities in range  $[0, 1]$ .

Once we have the predicted probability  $\hat{p}$  that `SalePrice > 130,000`, the *simplest* classification rule is:

$$\hat{y} = \begin{cases} 1, & \text{if } \hat{p} \geq 0.5, \\ 0, & \text{if } \hat{p} < 0.5 \end{cases}$$

That is, if the model's probability is at least 50%, predict membership in the "above \$130k" group (class 1); otherwise, predict class 0 (standard logistic "cutoff at 0.5").

```
In [90]: np.random.seed(42)
random_indices = np.random.choice(X_scaled.index, size=5, replace=False)
five_houses = X_scaled.loc[random_indices, selected_cols].copy()

five_houses_sm = sm.add_constant(five_houses, has_constant='add')
probabilities = final_model.predict(five_houses_sm)

print("Randomly selected house indices:", random_indices)
print("\nFeature values for these 5 houses:", five_houses)
print("\nPredicted probabilities:", probabilities)

# Demonstration of the formula for the first house
params = final_model.params
col_list = params.index.tolist()

house_1_index = random_indices[0]
house_1_data = five_houses_sm.loc[house_1_index]
house_1_features = house_1_data[col_list]

print("\n=====")
print(f"Detailed formula for House at index {house_1_index}:")

# z = beta0 + sum(beta_j * x_j)
z_value = 0.0
formula_parts = []
for col in col_list:
    beta = params[col]
    xval = house_1_features[col]
```

```

    product = beta * xval
    z_value += product
    if col == "const":
        formula_parts.append(f"{beta:.4f}")
    else:
        formula_parts.append(f"{beta:.4f}*({col}={xval:.4f})")

# p = 1 / (1 + exp(-z))
p_house_1 = 1 / (1 + np.exp(-z_value))

print("log-odds = ", " + ".join(formula_parts))
print(f"=> z_value = {z_value:.4f}")
print(f"=> Probability p = 1 / [1 + exp(-{z_value:.4f})] = {p_house_1:.4f}")
print("=====\n")

# p >= 0.5 => predict class 1, else 0
predicted_classes = (probabilities >= 0.5).astype(int)
print("Predicted classes (cutoff=0.5):", predicted_classes.values)

```

Randomly selected house indices: [ 892 1105 413 522 1036]

| Feature values for these 5 houses: |           |           |           | LotFrontage | LotArea   | OverallQual |
|------------------------------------|-----------|-----------|-----------|-------------|-----------|-------------|
| OverallCond                        | YearBuilt |           |           |             |           |             |
| 892                                | 0.006190  | -0.210750 | -0.071836 | 2.179628    | -0.273836 |             |
| 1105                               | 1.277754  | 0.174303  | 1.374795  | -0.517200   | 0.752907  |             |
| 413                                | -0.629592 | -0.156028 | -0.795151 | 0.381743    | -1.466183 |             |
| 522                                | -0.902070 | -0.552908 | -0.071836 | 1.280685    | -0.803768 |             |
| 1036                               | 0.869037  | 0.238646  | 2.098110  | -0.517200   | 1.183477  |             |

|      | YearRemodAdd | BsmtFinSF1 | TotalBsmtSF | 2ndFlrSF  | Fireplaces | WoodDeckS |
|------|--------------|------------|-------------|-----------|------------|-----------|
| 892  | 0.878668     | 0.481115   | 0.003581    | -0.795163 | -0.951226  | 0.780197  |
| 1105 | 0.491040     | 1.290428   | 0.924788    | 1.775997  | 2.152216   | 0.732311  |
| 413  | -1.689368    | -0.973018  | -0.112710   | -0.795163 | 0.600495   | -0.752176 |
| 522  | -1.689368    | -0.097907  | -0.121831   | 0.717284  | 2.152216   | -0.752176 |
| 1036 | 1.120936     | 1.268496   | 1.282782    | -0.795163 | 0.600495   | 1.067517  |

|      | ScreenPorch |
|------|-------------|
| 892  | -0.270208   |
| 1105 | -0.270208   |
| 413  | -0.270208   |
| 522  | -0.270208   |
| 1036 | -0.270208   |

Predicted probabilities: 892      0.946248  
1105    0.999843  
413     0.002129  
522     0.065903  
1036    0.999393  
dtype: float64

=====

Detailed formula for House at index 892:  
log-odds = 0.4496\*(LotFrontage=0.0062) + 2.4387\*(LotArea=-0.2107) + 1.2044\*(OverallQual=-0.0718) + 0.5936\*(OverallCond=2.1796) + 1.4551\*(YearBuilt=-0.2738) + 0.4779\*(YearRemodAdd=0.8787) + 0.6629\*(BsmtFinSF1=0.4811) + 1.3123\*(TotalBsmtSF=0.0036) + 1.6404\*(2ndFlrSF=-0.7952) + 0.6720\*(Fireplaces=-0.9512) + 0.2817\*(WoodDeckSF=0.7802) + 0.3614\*(ScreenPorch=-0.2702) + 3.8497  
=> z\_value = 3.0694  
=> Probability p = 1 / [1 + exp(-3.0694)] = 0.9556

=====

Predicted classes (cutoff=0.5): [1 1 0 0 1]

## Task 3.e

- Compute the classification table (confusion matrix) and calculate specificity and sensitivity.

- Provide verbal interpretation for the elements of the classification table and the performance measures (specificity, sensitivity, accuracy).

```
In [93]: X_sm = sm.add_constant(X_scaled[selected_cols], has_constant='add')
y_pred_prob = final_model.predict(X_sm)

y_pred_class = (y_pred_prob >= 0.5).astype(int)

cm = confusion_matrix(y, y_pred_class)
tn, fp, fn, tp = cm.ravel()

accuracy = (tp + tn) / (tp + tn + fp + fn)
sensitivity = tp / (tp + fn)
specificity = tn / (tn + fp)

print("Confusion Matrix:\n", cm)
print(f"True Negatives (TN) = {tn}, False Positives (FP) = {fp}")
print(f"False Negatives (FN) = {fn}, True Positives (TP) = {tp}")

print(f"\nAccuracy    = {accuracy:.4f}")
print(f"Sensitivity (Recall) = {sensitivity:.4f}")
print(f"Specificity = {specificity:.4f}")
```

Confusion Matrix:

```
[[349  27]
 [437 647]]
```

True Negatives (TN) = 349, False Positives (FP) = 27

False Negatives (FN) = 437, True Positives (TP) = 647

Accuracy = 0.6822

Sensitivity (Recall) = 0.5969

Specificity = 0.9282

The model is strong at identifying cheaper homes, but struggles with high-priced ones, leading to a high false negative rate — many expensive homes are incorrectly classified as being below \$130K. This suggests the model might need threshold adjustment or feature refinement to improve recall for expensive houses.

- $TP = 647$ : the model correctly identified 647 expensive houses as above 130K.
- $FN = 437$ : 437 expensive houses were misclassified as being below 130K.
- $FP = 27$ : 27 cheaper houses were mistakenly classified as above 130K.
- $TN = 349$ : 349 cheaper houses were correctly classified as below 130K.

The model is correct 68.2% of the time overall. Among expensive houses, it correctly identifies only 59.7% according to Sensitivity, meaning it misses many high-priced homes. Among cheaper houses, it correctly identifies 92.8%, meaning it is very good at detecting lower-priced homes according to Specificity.

## Task 3.f

It makes sense to change the threshold used for classification to improve performance. This can be done using the ROC curve.

- Plot this curve and determine the optimal threshold.
- Explain the concept of the curve and the AUC measure.

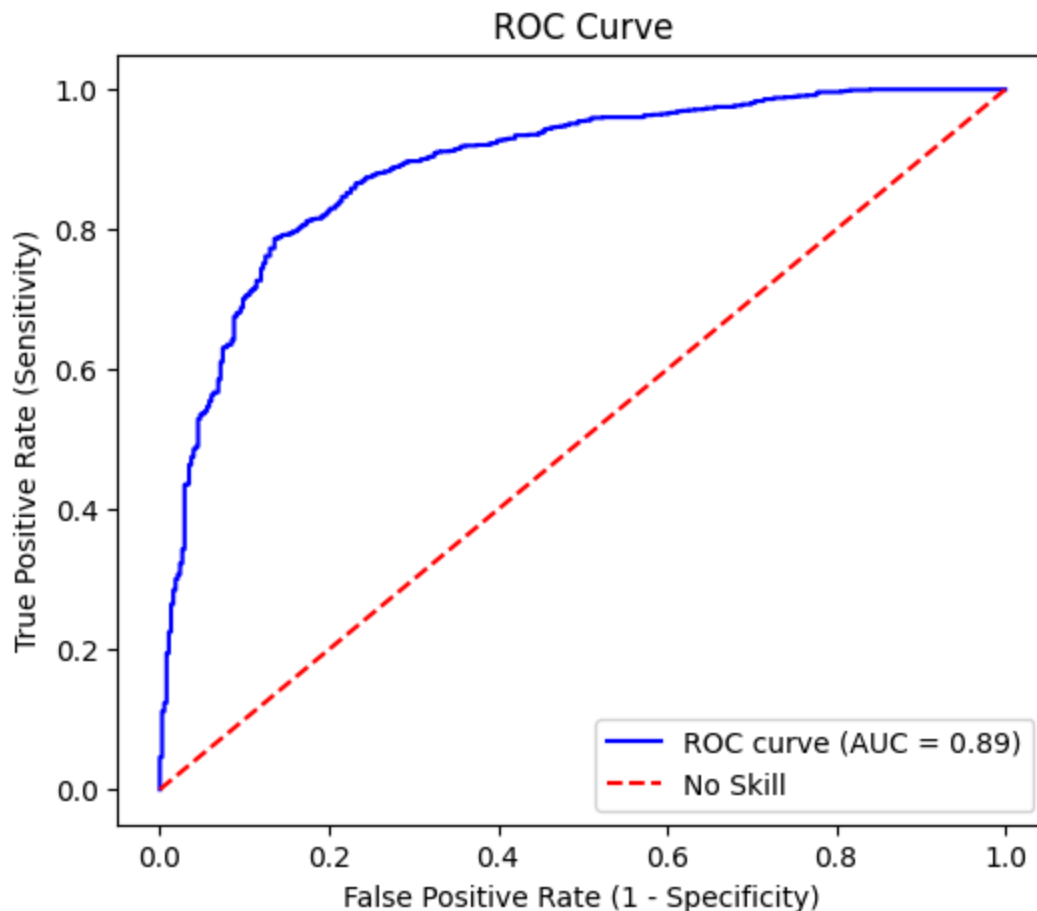
```
In [95]: X_sm = sm.add_constant(X_scaled[selected_cols], has_constant='add')
y_prob = final_model.predict(X_sm)

fpr, tpr, thresholds = roc_curve(y, y_prob)
roc_auc = roc_auc_score(y, y_prob)

print(f"AUC = {roc_auc:.4f}")

plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, label=f"ROC curve (AUC = {roc_auc:.2f})", color='blue')
plt.plot([0, 1], [0, 1], linestyle='--', color='red', label='No Skill')
plt.xlabel("False Positive Rate (1 - Specificity)")
plt.ylabel("True Positive Rate (Sensitivity)")
plt.title("ROC Curve")
plt.legend()
plt.show()
```

AUC = 0.8898



A common choice is the threshold that maximizes  $\text{tpr} - \text{fpr}$  (Youden's J statistic) or equivalently  $\text{tpr} + \text{tnr} - 1$ . Another approach is to pick the threshold that maximizes  $\text{tpr} + \text{tnr}$  sensitivity + specificity.

```
In [96]: # Determine an optimal threshold
youden_j = tpr - fpr
max_j_idx = np.argmax(youden_j)
best_thresh = thresholds[max_j_idx]

print(f"Optimal threshold by Youden's J: {best_thresh:.4f}")
print(f"At this threshold: TPR={tpr[max_j_idx]:.4f}, FPR={fpr[max_j_idx]:.4f}")

spec = 1 - fpr
sens_plus_spec = tpr + spec
max_ss_idx = np.argmax(sens_plus_spec)
best_thresh_ss = thresholds[max_ss_idx]
print(f"Optimal threshold by Sens+Spec: {best_thresh_ss:.4f}")
```

```
Optimal threshold by Youden's J: 0.1290
At this threshold: TPR=0.7869, FPR=0.1356
Optimal threshold by Sens+Spec: 0.1290
```

The optimal threshold (0.1290) is very close to zero, which means that the model is classifying houses as "expensive" even at a very low probability. This approach prioritizes sensitivity (recall) — correctly identifying expensive houses at the cost of more false positives (misclassifying cheaper houses as expensive).

## ROC-curve and AUC concepts:

The ROC-curve helps evaluate a classification model's performance across different probability thresholds. It plots the True Positive Rate (Sensitivity) against the False Positive Rate (1 - Specificity) for various thresholds.

- If we set the threshold *too high (near 1)*, the model predicts almost everything as *class 0*, leading to a low True Positive Rate but also a low False Positive Rate.
- If we set the threshold *too low (near 0)*, the model predicts almost everything as *class 1*, capturing all positives but also making many false positives.

The AUC measures how well the model separates positive and negative classes:

- $AUC = 1.0$  perfect classifier
- $AUC = 0.5$  useless random guessing
- $AUC = 0.89$  (in our case) strong performance, meaning the model ranks positives higher than negatives 89% of the time

## Task 3.g

- Recompute the classification table, sensitivity, and specificity for the new threshold. Provide an interpretation of the obtained values.
- Compare the results with the original values. Is the procedure now more conservative or less conservative?

```
In [97]: y_pred_new = (y_prob >= best_thresh).astype(int)

cm = confusion_matrix(y, y_pred_new)
tn, fp, fn, tp = cm.ravel()

accuracy = (tn + tp) / cm.sum()
sensitivity = tp / (tp + fn)
specificity = tn / (tn + fp)

print("Confusion Matrix at threshold = {:.4f}".format(best_thresh))
print(cm)
print(f"True Negatives (TN) = {tn}, False Positives (FP) = {fp}")
print(f"False Negatives (FN) = {fn}, True Positives (TP) = {tp}")

print(f"\nAccuracy    = {accuracy:.4f}")
print(f"Sensitivity   = {sensitivity:.4f}")
print(f"Specificity    = {specificity:.4f}")
```

Confusion Matrix at threshold = 0.1290

```
[[325  51]
 [231 853]]
```

True Negatives (TN) = 325, False Positives (FP) = 51

False Negatives (FN) = 231, True Positives (TP) = 853

Accuracy = 0.8068

Sensitivity = 0.7869

Specificity = 0.8644

### **Threshold = 0.5:**

Provides more conservative classification, as only assigns high-priced labels when the model is confident. We have higher specificity 92.8%, meaning fewer false positives (cheaper homes mistakenly labeled as expensive). But also lower sensitivity 59.7% means high false negatives (it misses many expensive houses).

### **Optimal threshold = 0.1290:**

More homes are labeled as expensive, even with lower confidence, with much higher sensitivity 78.7%, meaning fewer expensive houses are missed. We also have slightly lower specificity 86.4%, meaning more cheaper houses are misclassified as expensive. However the overall accuracy 80.7% is better compared to 68.2% at 0.5.

### **Which Threshold is Better?**

If catching expensive houses is more important (higher recall), the optimal threshold (0.1290) would be more appropriate. But if avoiding misclassifying cheaper houses as expensive is the priority, the default 0.5 threshold is better. Given that the optimal threshold significantly improves recall while keeping specificity reasonable, it is likely the better trade-off for this task.